

目录

第一部分 作品概述	3
1.1 功能与特性	3
1.2 应用领域	3
1.3 主要技术特点	3
1.4 主要性能指标	4
1.5 主要创新点	4
1.6 设计流程	6
第二部分 系统组成及功能说明	6
2.1 整体介绍	6
2.2 各模块介绍	7
(一) 基础模块设计	7
2.2.1 HDMI 发送模块	7
2.2.2 时钟生成模块	7
2.2.3 SDRAM 仲裁器模块	9
2.2.4 SD 卡控制模块	12
2.2.5 模式控制模块	15
2.2.6 UDP/IP 协议栈与 TEMAC 以太网 MAC 模块	19
2.2.7 子板控制模块	22
(二) 具体图形算法设计	24
2.2.8 模板匹配控制机制模块	24
2.2.9 帧差法运动检测模块	32
2.2.10 双目立体视觉模块	35
2.3 软件系统介绍	38
2.3.1 软件整体介绍	38
2.3.2 软件各模块介绍	39
第三部分 完成情况及性能参数	45
3.1 整体介绍	45
3.2 工程成果	49
3.2.1 电路成果	53
3.2.2 软件成果	54
3.3 特征成果	57
第四部分 总结	61
4.1 可扩展之处	61
4.1.1 图像处理能力扩展	61
4.1.2 网络通信协议扩展	61
4.1.3 存储系统扩展	61
4.1.4 显示输出扩展	61
4.1.5 智能控制扩展	61
4.2 心得体会	62
第五部分 参考文献	62
第六部分 附录	63

双目视觉处理系统

摘要

本项目基于安路 EG4S20BG256 构建“主板+扩展板+上位机”三端协同的视频综合处理系统。主板集成 RGMII+UDP 协议栈、SDRAM 帧缓存与 HDMI 输出，形成两条可动态切换的数据通路：其一，承接上位机/扩展板的图像或视频，经“UDP→SDRAM→HDMI”实时显示；其二，支持 SD 卡与双目 OV5640 采集，经 SDRAM 缓存后封装为 UDP 帧回传。视频分片采用“32 字节包头+636 字节数据、每帧 1450 包”，包头含魔数、宽高、总字节、偏移、帧/包序等字段，并可携带模板匹配结果。双目分片采用“32 字节包头 + 数据区（RGB 联合模式 848B / 测距二值模式 424 包），每帧 1450 包”。系统支持 640×480/1280×720/1920×1080 三档分辨率与像素时钟重配置，图像算法流水线涵盖二值化、腐蚀、膨胀、开运算、Sobel 算子、拉普拉斯变换、图像锐化算法、浮雕效果演示、帧差法、颜色识别与边框叠加、32×32 二值化模板匹配与基于 RGB 阈值的双目测距算法。双目视觉由双路独立 OV5640 同时启动并采集，经大容量异步 FIFO 与行/像素级延时配准完成帧级同步与左右目对齐，在图像域基于可远程配置的 RGB 阈值提取目标并求取左右目边界框中心坐标，以视差 $d=x_l-x_r$ 进入定点流水线按 $D=(B \cdot f)/d$ 实时计算距离（cm，8 位量化），结果经 CDC 同步到 UDP 域并随视频/控制数据回传至上位机用于显示与记录。跨时钟可靠性通过两级打拍/异步 FIFO、SDRAM 读写仲裁与地址隔离保障，分辨率切换配合像素时钟复位确保稳定。扩展板以按键/拨码开关与 LED/数码管为人机界面，可一键切换业务模式、自动下发 0x0003/0x0004 命令，完成 SD 卡 BMP→UDP 分片发送并以本地 HDMI 回显实现双板闭环控制。上位机（PyQt5+OpenCV）提供分辨率/阈值/模板等远程配置、单/双目回显、带宽与帧率监测及图像抓拍。实测单目视频流在 640×480 的分辨率下帧率最高能达到 60FPS、端到端时延 200ms 以内、UDP 有效带宽 510Mbps；双目目视频流在双 640×480 的分辨率下帧率最高能达到 50FPS、UDP 有效带宽 550Mbps。资源占用与时序裕量满足目标器件与应用需求，鲁棒性良好。方案使用“网络远程配置+图像流水线+资源复用”的设计思想，具备高可扩展性，适用于工业检测、安防监控与教学实验等场景。

第一部分 作品概述

1.1 功能与特性

系统实现双模式视频链路：UDP→SDRAM→HDMI 显示与 SD/OV5640→SDRAM→UDP 回传；支持 VGA/720p/1080p 与像素/5x 时钟重配置。自定义 UDP 分片（32B 包头+数据，1450 包/帧）：单目 636B/包，双目 RGB 848B/包，测距二值 424B/包，包头可携带模板匹配与测距结果。上位机/扩展板可远程配置分辨率、处理/识别模式与阈值，扩展板可替代上位机完成 LED/数码管、图片下发、SD 回传与单目接收并本地 HDMI 回显。图像算法含二值、腐蚀/膨胀/开运算、Sobel/拉普拉斯、浮雕、帧差与边框叠加；识别含阈值目标+方框、32×32 模板匹配、双目测距 ($D=(B \cdot f)/d$)。为保证低延迟与可靠性，采用 CDC 两级打拍+异步 FIFO、SDRAM 仲裁与地址隔离、首字节缓存防错位；测距模式半带宽传输，读写解耦提升实时性，整体模块化、可扩展。

1.2 应用领域

- (1) 工业检测与分拣：基于阈值识别/模板匹配与双目测距，完成缺陷检测、尺寸测量、颜色分拣；UDP 远程参数下发，支持 VGA/720p/1080p 快速换型上线。
- (2) 安防与边缘监控：帧差+边框叠加实现运动目标标注与事件回传；低时延传输与断网本地显示，提高复杂网络下的可靠性与可用性。
- (3) 机器人/AGV/无人车：双目测距用于避障与场景理解；上位机/扩展板远程配置阈值与模式，便于现场快速标定与任务切换。
- (4) 教学与竞赛演示：完整链路（采集/处理/传输/显示）与可插拔算法流水线，直观呈现 UDP 协议、跨时钟域和 SDRAM 仲裁，便于实验与扩展创新。

1.3 主要技术特点

(1) 多模式集成技术:将 LED 控制、图像采集、处理、显示和网络传输等多种功能模块集成到单一 FPGA 平台，通过状态机设计实现工作模式无缝切换，采用模块化架构提高系统灵活性和可维护性，实现硬件资源高效复用。

(2) 双目视觉处理技术:支持两个 OV5640 摄像头同步采集图像，通过 SDRAM 地址空间划分实现双路数据独立存储，开发了专门的图像同步与合并算法，为 3D 视觉应用、深度感知和立体匹配提供了硬件基础。

(3) 实时图像处理技术:在 FPGA 上实现了 RGB 转灰度、动态阈值二值化、

腐蚀膨胀等形态学处理算法,采用流水线架构提高处理效率,支持 30fps 实时视频流处理, 处理参数可通过网络动态配置, 处理延迟控制在 50ms 以内。

(4) 网络化控制技术:基于 UDP 协议栈实现了完整的网络通信系统, 支持远程参数配置、模式切换和图像传输, 开发了专用命令解析模块, 实现多路数据流复用传输, 构建了灵活的远程控制架构。

(5) 资源优化技术:通过 SDRAM 仲裁器实现多模块共享存储资源, 采用异步 FIFO 解决跨时钟域数据传输问题, 优化数据流和时序设计提高系统吞吐量, 合理分配 FPGA 逻辑资源, 在有限硬件平台上实现了复杂功能集成。

1.4 主要性能指标

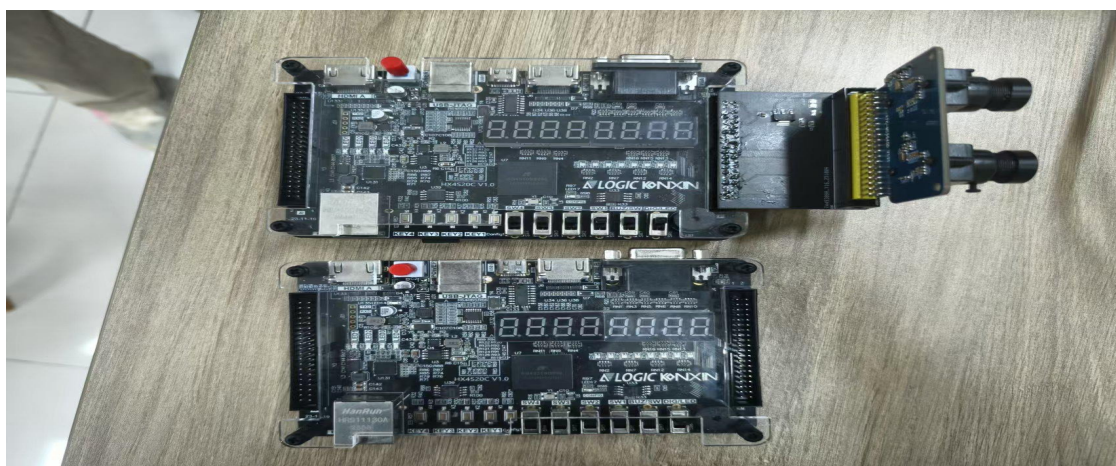
性能指标	参数值	说明
图像分辨率	640x480	全局 640x480, 部分功能支持 640x480、1280x720、1920x1080 多档分辨率
帧率	56fps	640x480 RGB888 分辨率下单目实时传输帧率
网络带宽	510Mbps	双目视频传输模式下最高稳定传输速率
处理延迟	<200ms	图像采集到处理完成的端到端延迟
LUT 占用	18397	主板 11666 个, 子板 6731 个
REG 占用	14450	主板 8902 个, 子板 5548 个
BRAM9K	66	主板 52 个, 子板 14 个
BRAM32K	8	主板 8 个
DSP	13	主板 10 个, 子板 3 个
PLL	6	主板 3 个, 子板 3 个

1.5 主要创新点

- (1) 单异步 FIFO + 编帧同步的双目采集方案: 左右目分别经异步 FIFO 桥接, 结合行/像素级延时与帧序号对齐, 实现低资源的双目同步, 占用少且稳定。
- (2) 降采样帧差的运动检测: 在降采样域进行时域帧差与阈值判定, 显著降低图像存储资源, 使用 BRAM 代替 SDRAM 进行缓存。
- (3) 上位机可配置的模板匹配: 32×32 二值模板+汉明距离, 模板与阈值在线热

更新； μs 级处理、LUT 实现，匹配结果嵌入 UDP 包头，无需额外控制链路。

- (4) 双目测距全链路优化：RGB 阈值筛选+边界框中心视差 d ，按 $D=(B \cdot f)/d$ 实时定点计算（cm，8 位）；测距二值模式采用 424B/包（32B 头+424B 数据），带宽减半、延迟更低；结果随包头回传，上位机 0x0009/0x00F5 在线调参与模式切换。
- (5) 三板协同：上位机+主板+扩展板协同联动；扩展板可独立完成 LED/数码管控制、SD 卡 BMP→UDP 分片发送、单目接收与本地 HDMI 回显，并自动下发 0x0003/0x0004 模式指令。
- (6) 以太网协议栈与打包策略优化：固定 1450 包/帧、32B 包头；首字节缓存防错位；app_tx_request 统一仲裁；双目模式动态包长（RGB 848B / 测距 424B），在保证兼容性的同时优化带宽与实时性。
- (7) 多分辨率与像素时钟重配置：VGA/720p/1080p 像素/5x 时钟动态切换，配合像素时钟复位确保锁相与显示稳定；读请求与时序自适应，切换无死锁。
- (8) SDRAM 地址隔离与读写仲裁：项目内多路径互不干扰；上行/下行解耦，缓冲深度与突发长度匹配视频流水，降低抖动与链路峰值拥塞。
- (9) 人机交互与可观测性：PyQt5 UI 支持分辨率/阈值/模板热配置、单/双目回显、带宽/FPS 实时监测、抓拍；提供双目字节偏移校正与日志/超时丢帧策略，调试体验友好、问题可定位。
- (10) 可扩展与工程化：协议/命令与算法流水线可插拔，易引入新分辨率与算子；关键模块采用分层与复用思路（如模板匹配用分布式 RAM、双目包头复用字段），在低逻辑占用下保持可维护性与扩展空间。



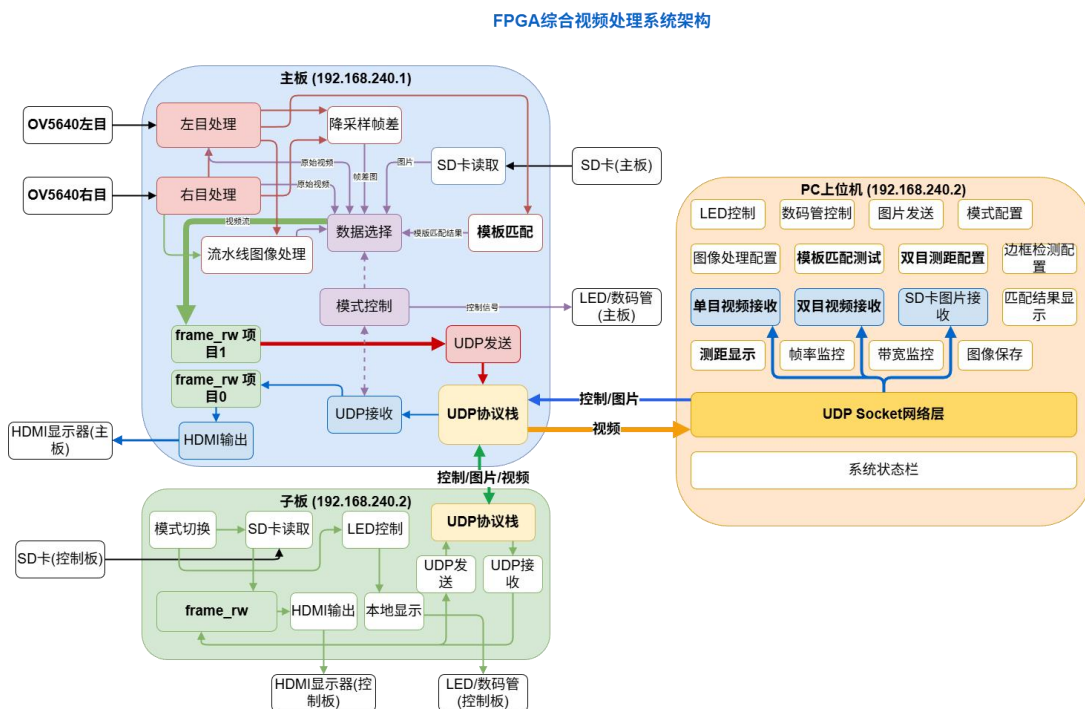
1.5.1 系统全貌图

1.6 设计流程

- (1) 分功能实现: 依次完成 LED/数码管控制→上位机发送 BMP 图片数据到 hdmi 显示→SD 卡读取图片传输到上位机→OV5640 视频流传输到上位机。
- (2) 分阶段集成: 先集成 LED/数码管控制和上位机发送 BMP 图片、再集成 SD 卡/OV5640 读取图片传输到上位机, 逐步打通端到端链路。
- (3) 总集成: 合并全链路, 并同步迭代上位机 UI/命令与协议。
- (4) 子板工程: 实现前四项的上位机替代(控制、发图、接收、回显), 闭环调。
- (5) 双目扩展: 新增双目采集、同步配准与分片打包, 上位机双窗显示。
- (6) 算法接入: 按序加入二值/形态/边缘/浮雕、帧差、模板匹配与双目测距。
- (7) 优化验收: 在各分辨率下验证帧率/时延/带宽; 完善 CDC、SDRAM 仲裁与复位策略。

第二部分 系统组成及功能说明

2.1 整体介绍



2.2 各模块介绍

（一）基础模块设计

2.2.1 HDMI 发送模块

HDMI 发送模块核心是将并行视频数据转换为 HDMI 标准串行信号输出的，我们将工作流程分为四步：先接收 24 位 RGB 并行数据与同步信号，再通过 TMDS 编码将 8 位数据转为 10 位以增强直流平衡和时钟恢复能力，接着将并行数据串行化，最后通过 4 个 TMDS 差分通道输出。模块由五大关键部分构成：HDMI 顶层模块负责连接 VGA 与 HDMI 接口；DVI 发射器实现 VGA 信号到 TMDS 信号的转换；TMDS 编码器执行 8B/10B 编码算法；视频时序生成器生成显示所需时序信号；彩条生成器提供测试调试用的彩条信号。技术参数上，视频支持 24 位真彩色与 640×480 至 1920×1080 多种分辨率，TMDS 采用 3 个数据通道 + 1 个时钟通道的 8B/10B 编码，物理接口为 HDMI Type A，单通道最大传输速率 3.4Gbps。功能特性包括多分辨率动态切换、低延迟真彩色处理、差分传输保障信号完整性、兼容 HDMI/DVI 且向下兼容，还具备低电压操作与动态功耗管理的低功耗设计，是我们目前的最优解。

2.2.2 时钟生成模块

（1）功能实现

time_top 模块是一个多功能时钟生成器，主要负责为整个 FPGA 系统提供各种频率和相位的时钟信号。其核心功能包括：

- 1.多频率时钟生成：通过 PLL 和分频器生成多种频率的时钟。
- 2.多分辨率支持：提供 640×480、720p 和 1080p 三种分辨率的像素时钟。
- 3.时钟相位控制：生成具有不同相位偏移的时钟（0 度、90 度、180 度）。
- 4.动态时钟选择：根据输入控制信号选择不同的时钟输出。

（2）设计架构

模块采用分层架构设计，主要包含以下几个部分：

1. PLL 时钟生成：使用两个独立 PLL（`pll\_core` 和 `video\_pll`）生成基础时钟
2. 时钟选择逻辑：通过多路选择器根据分辨率选择信号输出相应时钟
3. 时钟分频器：使用计数器实现进一步的时钟分频
4. 时钟同步处理：实现控制信号的时钟域同步，防止亚稳态

关键参数配置

1. 输入输出接口

输入信号：

- (1) `clk_50m`: 50MHz 参考时钟输入
- (2) `rst_n`: 异步复位信号（低电平有效）
- (3) `resolution_sel[1:0]`: 分辨率选择信号

输出信号：

- (1) 标准时钟: 125MHz(0 度×2、90 度)、25MHz、50MHz(0 度、180 度)、12.5MHz、1.25MHz
- (2) 视频时钟: `video_clk_out`(像素时钟)、`video_clk_5x_out`(5 倍像素时钟)
- (3) 状态信号: `pll_locked`、`video_pll_locked`

2. PLL 配置参数

`pll_core`:

- (1) 输入频率: 50MHz
- (2) 输出时钟: 125MHz(0° /90°)、50MHz(0° /180°)、25MHz
主要用于系统控制和 640×480 分辨率支持

`video_pll`:

- (1) 输入频率: 50MHz
- (2) 输出时钟:
720p 模式: 74.25MHz(像素时钟)和 371.25MHz(5x 时钟)
1080p 模式: 148.5MHz(像素时钟)和 742.5MHz(5x 时钟)

3. 分频器配置

- (1) 12.5MHz 分频器: 25MHz 输入，2 分频实现。
- (2) 1.25MHz 分频器: 12.5MHz 输入，5 分频实现。

工作原理

1. 时钟生成过程：

以 50MHz 外部时钟为参考，通过两个 PLL 生成所需的高频时钟
对于更低频率的时钟需求，通过数字分频器实现

2. 分辨率切换机制:

外部输入的`resolution\_sel`信号首先通过同步寄存器处理,防止亚稳态
根据同步后的选择信号,组合逻辑多路器选择对应的时钟输出

当 resolution_sel=0(640×480)时,选择 25MHz/125MHz

当 resolution_sel=1(720p)时,选择 74.25MHz/371.25MHz

当 resolution_sel=2(1080p)时,选择 148.5MHz/742.5MHz

3. 复位与使能控制:

异步复位信号控制整个模块的初始状态

所有分频器的工作受 PLL 锁定信号(pll_locked)控制,确保时钟稳定后才开始工作。

4. 时钟域同步:

为避免亚稳态,分辨率选择信号在使用前先同步到系统时钟域。

系统作用如下:

该时钟模块是整个 FPGA 系统的"心脏",为各个功能模块提供精确的时钟信号,确保:

1. 视频处理模块获得正确的像素时钟。
2. 网络通信模块获得所需的 125MHz 以太网时钟。
3. 控制逻辑和低速外设获得合适频率的时钟信号。
4. 所有模块在正确的时序约束下协调工作。

通过动态时钟选择机制,系统能够灵活支持多种分辨率,提高了硬件资源的利用率。

2.2.3 SDRAM 仲裁器模块

一、基本概念与作用

SDRAM 仲裁器是连接多个 SDRAM 访问请求源与单一 SDRAM 控制器之间的关键接口模块。在复杂的 FPGA 系统中,多个功能模块(如图像采集、图像处理、数据传输等)往往需要同时访问 SDRAM 资源,而 SDRAM 控制器同一时间只能处理一个访问请求。因此,SDRAM 仲裁器的核心作用是:

1. 资源分配管理:协调多个主机对共享 SDRAM 资源的访问请求
2. 时序冲突避免:确保不同模块的访问请求不会在时间上冲突
3. 带宽优化分配:根据系统需求合理分配内存带宽
4. 地址空间管理:管理不同模块的内存地址分配,避免地址冲突

二、主要功能实现

从当前工程的 sdran_arbiter 模块实现来看,它采用了模式切换式仲裁机制,主要功能包括:

1. 双工程切换:通过 mode_project 信号在两个独立工程(bp12_2 和 q3v3)之间进行切换。
2. 地址空间隔离:使用地址偏移量(`ADDR\_OFFSET\_PROJECT1`)将两个工程的

SDRAM 访问空间完全隔离。

3. 读写接口仲裁：分别对读请求和写请求进行仲裁，切换相应的控制信号和数据信号。

4. 数据回传分发：将 SDRAM 返回的数据正确分发到对应的请求工程。

写接口仲裁示例

信号名称	赋值表达式	描述
sdrAm_App_wr_en	assign sdrAm_App_wr_en = mode_project ? prj1_App_wr_en : prj0_App_wr_en;	根据模式选择信号 (mode_project) 切换两个工程的 SDRAM 写使能信号，实现双工程写请求仲裁
sdrAm_App_wr_addr	assign sdrAm_App_wr_addr = mode_project ? (prj1_App_wr_addr + ADDR_OFFSET_PROJECT1) : prj0_App_wr_addr;	根据模式选择信号切换写地址，其中工程 1 (q3v3) 的地址叠加偏移量 (ADDR_OFFSET_PROJECT1=21'd307200)，实现两个工程的 SDRAM 地址空间隔离

三. 常见仲裁算法及比较

1. 优先级仲裁(Priority Arbitration)

工作原理：为每个请求源分配固定优先级，高优先级请求总是优先于低优先级请求

优点：

实现简单，控制逻辑开销小。

确保关键模块始终获得及时访问。

缺点：

可能导致低优先级模块的饥饿问题。

内存带宽利用率不高。

2. 轮询仲裁(Round Robin Arbitration)

工作原理：按固定顺序依次为每个请求源分配访问权限，确保每个请求源都有机会访问。

优点:

公平性好, 避免饥饿问题。

带宽分配均衡。

缺点:

关键请求可能无法得到及时响应。

实现复杂度较高。

四. 加权轮询(Weighted Round Robin)

工作原理: 结合优先级和轮询的优点, 为不同请求源分配不同的权重, 按权重比例分配访问时间。

优点:

兼顾公平性和优先级。

可以根据实际需求灵活调整带宽分配。

缺点:

实现复杂度高。

参数调整需要根据系统特性进行优化。

五. 模式切换式仲裁(Mode Switch Arbitration)

工作原理: 如当前工程所采用的, 通过外部控制信号在不同工作模式之间完全切换, 某一时刻只有一个模式下的模块能访问内存。

优点:

实现非常简单, 几乎没有仲裁延迟。

模块间地址空间完全隔离, 避免冲突。

适合于功能明确分离的多工程系统。

缺点:

灵活性较差, 不能实现真正的并发访问。

在模式切换期间可能有短暂的内存访问中断。

六. 内存带宽优化与延迟降低

带宽优化策略如下:

1. 地址空间隔离: 通过预分配不同地址区域给不同模块, 避免地址重叠导致的冲突和额外开销。
2. 突发传输优化: 鼓励长突发传输而非多个单字节传输, 减少命令开销。
3. 请求合并: 将短时间内的多个相邻地址请求合并为一次突发传输。
4. 预取机制: 根据访问模式预测并提前获取可能需要的数据。

延迟降低方法如下:

1. 仲裁延迟最小化: 简单的仲裁逻辑如当前工程使用的模式切换可以将仲裁延

迟降至最低。

2. 请求缓冲：为每个请求源提供请求队列，避免请求被拒绝导致的重试开销。
3. 优先级动态调整：根据系统状态动态调整请求优先级。
4. 避免频繁切换：减少不同请求源之间的频繁切换，降低命令切换开销。

七.总结

SDRAM 仲裁器是多模块共享 SDRAM 资源的关键组件，其设计直接影响系统性能和资源利用率。本工程采用的模式切换式仲裁虽然在灵活性上有所限制，但在实现复杂度、可靠性和资源占用方面具有明显优势，非常适合该系统的双工程架构需求。在更复杂的系统中，可能需要考虑优先级仲裁或轮询仲裁等更灵活的机制，但会相应增加实现复杂度和资源消耗。

2.2.4 SD 卡控制模块

一.核心功能

1. SD 卡初始化与控制：通过 SPI 接口与 SD 卡通信，完成初始化、读写等基本操作
2. BMP 文件搜索与识别：在 SD 卡中搜索符合指定宽度的 BMP 格式文件
3. BMP 文件解析：读取并解析 BMP 文件头信息，提取图像参数（宽度、高度、位深度等）
4. 图像数据处理：支持 24 位和 32 位 BMP 图像格式，自动处理字节对齐和填充字节
5. 数据格式转换：将 BMP 格式的像素数据（BGR 存储）转换为系统所需格式（RGB+0 填充）
6. 状态管理与指示：提供状态编码，显示当前操作阶段（初始化、等待、搜索、读取）

二、模块架构与组成

SD 卡控制模块由三个主要模块组成

模块名称	子模块
sd_card_bmp	bmp_read
	sd_card_top
	ax_debounce
bmp_read	BMP 文件解析与图像处理模块
sd_card_top	sd_card_sec_read_write: 扇区读写控制模块
	sd_card_cmd: SD 卡命令处理模块
	spi_master: SPI 通信主控制器
ax_debounce	按键消抖模块

模块结构表

模块层次关系如下：

1. 应用层：bmp_read 负责 BMP 文件解析和图像处理逻辑
2. 中间层：sd_card_sec_read_write 处理扇区读写操作，管理 SD 卡命令序列
3. 协议层：sd_card_cmd 实现 SD 卡命令协议，如初始化、读写命令等
4. 物理层：spi_master 实现 SPI 通信协议，直接与 SD 卡硬件交互

三、工作原理

初始化流程

1. 系统复位后，SD 卡模块进行初始化
2. SD 卡初始化完成后，系统进入等待状态（state_code=1）
3. 按键按下时（key 信号下降沿），触发 BMP 文件搜索过程

BMP 文件搜索过程

1. 从起始扇区（默认 16000）开始，每隔 8 个扇区（4KB）搜索 BMP 文件头。
2. 解析 54 字节 BMP 文件头，验证文件标识符（BM）。
3. 提取文件长度、像素数据偏移量、图像宽度、高度和位深度信息。
4. 检查图像宽度是否与指定宽度匹配，以及位深度是否为 24 位或 32 位。
5. 如果找到匹配的 BMP 文件，进入读取状态；否则继续搜索。

四. 图像数据读取与处理

1. 搜索成功后，发送 write_req 请求，等待 write_req_ack 响应
2. 开始读取像素数据，从 data_offset 指定的偏移量开始
3. 根据 BMP 文件的位深度（24 位或 32 位）进行不同处理：
 - 24 位 BMP：每 3 个字节组成一个像素，按 BGR 顺序存储
 - 32 位 BMP：每 4 个字节组成一个像素，按 BGRA 顺序存储（忽略 Alpha 通道）
4. 自动处理 BMP 文件的行对齐填充字节
5. 将 BGR 格式转换为 RGB 格式，并按要求格式输出：{R, G, B, 0}

状态机设计

状态名称	状态描述
S_IDLE	空闲状态，等待按键触发
S_FIND	BMP 文件搜索状态
S_READ_WAIT	等待写入请求确认
S_READ	图像数据读取状态
S_END	读取结束状态

五、技术特点

1. 自适应 BMP 格式支持
 - (1) 支持标准 24 位和 32 位 BMP 图像格式
 - (2) 自动识别文件头中的数据偏移量，不硬编码 54 字节的头大小
 - (3) 自动计算图像总像素数和每行字节数，适应不同尺寸图像
2. 优化的数据处理机制
 - (1) 采用字节计数器管理 RGB 数据组装，确保数据准确性
 - (2) 实现行内像素计数，智能跳过 BMP 文件中的行对齐填充字节
 - (3) 根据实际像素数而非文件大小判断读取结束，提高兼容性
3. 可靠的错误处理
 - (1) 通过状态编码提供直观的操作反馈
 - (2) 对无效像素范围进行边界检查，避免错误数据输出
 - (3) 搜索机制具有跳步功能，加速文件查找
4. 与系统兼容的数据格式
 - (1) 输出格式设计与摄像头数据格式保持一致：{R, G, B, 0}
 - (2) 实现 BMP 文件 BGR 存储到系统 RGB 处理的格式转换

六、总结

SD 卡控制模块(sd_card_bmp)是一个设计良好的 SD 卡读写与 BMP 图像解析模块，能够高效地从 SD 卡中搜索、读取和处理 BMP 格式图像文件。该模块采用分层设计，结合了 SPI 通信、SD 卡协议和 BMP 文件格式解析，具有良好的兼容性和可靠性。通过灵活的接口设计，该模块可以方便地集成到各种需要图像处理的 FPGA 系统中。

2.2.5 模式控制模块

一、模块概述与设计目标

udp_mode_ctrl 是一个高度集成的模式控制与命令解析模块，主要负责解析上位机通过 UDP 协议发送的控制命令，并根据命令内容动态切换系统的工作模式、配置参数和控制外设功能。该模块是连接上层控制软件与底层硬件功能的核心桥梁。

设计目标如下：

1. 多工程兼容：支持 bp12_2 和 q3v3 两个不同工程的模式切换
2. 模式控制：提供 LED、数码管、SD 卡、摄像头等多种功能模式
3. 图像处理：支持多种实时图像处理算法的选择和配置
4. 多接口控制：同时支持 UDP 远程控制和本地按键控制
5. 参数动态配置：允许实时调整二值化阈值等处理参数
6. 高扩展性：预留分辨率切换等扩展功能接口

二、架构设计与核心组件

udp_mode_ctrl 模块采用层次化设计，包含三个主要功能层：

1. 通信接口层：处理 UDP 数据接收和按键输入
2. 命令解析层：解析命令格式，验证参数有效性
3. 控制输出层：生成各种控制信号，驱动下游模块

核心组件如下

1. 状态机控制器

状态名称	状态值 (二进制)	状态描述
IDLE	2' d0	空闲状态，等待 UDP 数据包或按键输入触发新的命令接收流程
RECV_DATA	2' d1	数据接收状态，按顺序接收并存储 UDP 命令字节（支持 2 字节/4 字节命令格式）
PROCESS	2' d2	命令处理状态，解析命令内容、验证参数有效性（如阈值范围），生成控制信号驱动下游模块

状态机实现了 UDP 命令的接收和处理流程控制，包括空闲等待、数据接收和命令处理三个主要状态。

2.控制输出接口

信号名称	类型	描述
mode_project	output reg	工程模式控制:0 表示 bp12_2 工程,1 表示 q3v3 工程
mode_q3v3_internal	output reg	q3v3 工程内部模式控制: 0 表示 SD 卡模式, 1 表示摄像头模式
mode_bp12_led_seg	output reg	bp12_2 工程内部模式控制: 0 表示 LED 模式, 1 表示数码管模式

三、UDP 通信协议规范

1. 支持的命令集

命令类型	命令代码	功能描述
工程模式切换命令	0x0001	LED 模式
工程模式切换命令	0x0002	数码管模式
工程模式切换命令	0x0003	SD 卡模式 + 触发读取
左目摄像头命令	0x0004	左目原始图像输出
左目摄像头命令	0x0005	左目二值化图像输出
左目摄像头命令	0x0006	左目腐蚀图像输出
左目摄像头命令	0x0007	左目开运算图像输出
左目摄像头命令	0x000A	左目浮雕效果图像输出
左目摄像头命令	0x0008	左目拉普拉斯锐化图像输出
左目摄像头命令	0x000C	左目 Sobel 边缘检测图像输出
左目摄像头命令	0x000D	左目帧差法处理
右目摄像头命令	0x0010	右目原始图像输出
右目摄像头命令	0x0012	右目二值化图像输出
右目摄像头命令	0x0013	右目腐蚀图像输出
右目摄像头命令	0x0014	右目开运算图像输出
右目摄像头命令	0x0015	右目浮雕效果图像输出
右目摄像头命令	0x0016	右目拉普拉斯锐化图像输出
右目摄像头命令	0x0017	右目 Sobel 边缘检测图像输出
双目模式命令	0x0011	双目摄像头原始图像输出
参数配置命令	0x0008	设置二值化高低阈值
功能开关命令	0x0009	切换边缘检测使能（使能/禁用切换）
分辨率控制命令	0x0030	设置分辨率为 640×480
分辨率控制命令	0x0031	设置分辨率为 1280×720（720p）
分辨率控制命令	0x0032	设置分辨率为 1920×1080（1080p）

四、工作流程与状态转换

1. 命令接收与处理流程

初始化阶段：

模块复位后，设置默认工作参数（默认 bp12_2 工程，LED 模式等）

状态机进入 IDLE 状态，等待 UDP 数据包或按键输入

数据接收阶段：

收到 UDP 数据时，首先检查数据包长度（只处理 2 字节或 4 字节命令）

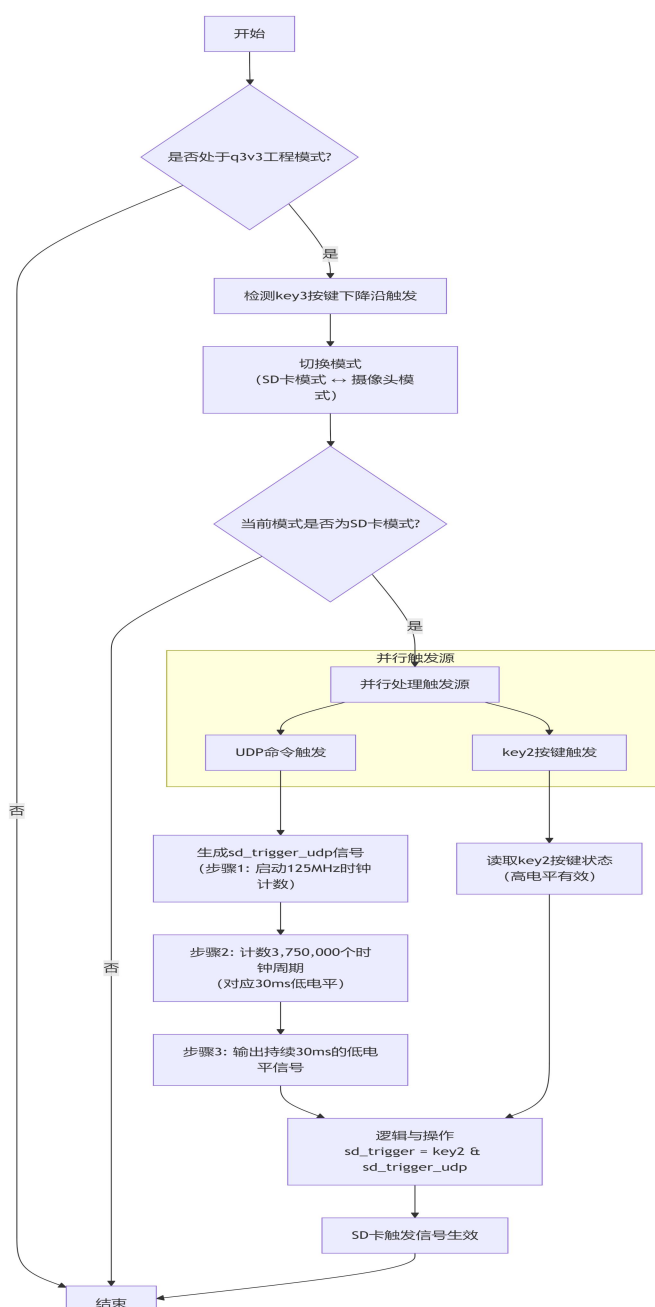
进入 RECV_DATA 状态，按顺序接收并存储命令字节和参数字节

对于 2 字节命令，接收完成后直接进入处理阶段

对于 4 字节命令，完整接收 2 字节命令和 2 字节参数后进入处理阶段。

命令处理阶段：根据命令类型更新相应的控制信号对参数进行有效性验证（如二值化阈值）处理完成后，状态机返回 IDLE 状态等待下一条命令。

2. 按键控制流程



六、错误处理机制

udp_mode_ctrl 模块实现了以下错误处理机制：

1. 命令过滤：仅处理数据包长度为 2 字节或 4 字节的 UDP 命令对于未知命令，保持当前状态不变。
2. 参数验证：二值化阈值配置时，确保低阈值小于高阈值：`if (param1 < param2)` 如果参数不合法，保持原有阈值不变，不产生任何错误输出。
3. 状态机保护：状态机采用 default 分支，防止状态异常复位时对所有寄存器进行初始化，确保系统启动状态一致
4. 按键抖动防护：实现按键边沿检测，避免抖动引起的误操作 SD 卡触发信号采用脉冲展宽技术，确保操作可靠性。

总结来看，udp_mode_ctrl 模块是一个设计良好的控制中枢，通过灵活的命令解析和控制机制，实现了对多工程、多功能的统一管理。其丰富的接口和控制能力使其成为整个系统的核心协调模块，为上层应用提供了便捷的控制途径。

2.2.6 UDP/IP 协议栈与 TEMAC 以太网 MAC 模块

一、概述

UDP/IP 协议栈与 TEMAC (Tri-Mode Ethernet MAC) 是网络通信系统的核心组件，二者协同工作实现从应用层数据到物理层信号的完整传输链路。其中，UDP/IP 协议栈负责网络层 (IP) 和传输层 (UDP) 的功能，完成数据的封装与解封装；TEMAC 作为以太网媒体访问控制控制器，承担数据链路层功能，管理以太网帧的收发及与物理层的交互，最终通过 RGMII 接口实现网络数据的物理传输。

二、UDP/IP 协议栈

UDP/IP 协议栈是一套采用模块化设计的网络通信协议集合，涵盖网络层到传输层的核心功能，能够为应用层提供可靠的网络通信服务，支持数据封装、传输与接收的全流程处理，具体包括将应用层数据按协议规范添加 UDP 头、IP 头及 MAC 头，封装为以太网可传输的数据包，同时在接收以太网帧后逐层解析 MAC 头、IP 头、UDP 头以提取有效应用层数据，还可实现本地 UDP 端口号、IP 地址、MAC 地址的配置与管理，通过握手信号调节数据传输速率以避免数据溢出，并集成基本错误检测机制，保障数据传输的可靠性。

三、接口定义

(1) 配置参数

DEVICE: 目标 FPGA 器件类型；

LOCAL_UDP_PORT_NUM: 本地 UDP 端口号；

LOCAL_IP_ADDRESS: 本地 IP 地址；

LOCAL_MAC_ADDRESS: 本地 MAC 地址。

(2) 应用层接口

发送方向：app_tx_*系列信号（应用层→UDP/IP 协议栈）；

接收方向：app_rx_*系列信号（UDP/IP 协议栈→应用层）。

3. TEMAC 接口

接收方向：temac_rx_*系列信号（TEMAC→UDP/IP 协议栈）；

发送方向：temac_tx_*系列信号（UDP/IP 协议栈→TEMAC）。

(3) 数据处理流程

1. 封装流程（应用层→物理层）

应用层提供原始数据；

UDP 层添加 UDP 头（含源端口、目标端口、长度、校验和）；

IP 层添加 IP 头（含源 IP、目标 IP、协议类型等）；

MAC 层添加 MAC 头（含源 MAC、目标 MAC、类型 / 长度字段），形成完整以太网帧。

2. 解封装流程（物理层→应用层）

接收来自 TEMAC 的以太网帧；

解析 MAC 头，校验目标 MAC 地址是否为本机；

解析 IP 头，提取 IP 地址信息并完成路由判断；

解析 UDP 头，根据端口信息定位目标应用；

提取应用层有效数据，传递给上层应用。

四、TEMAC 以太网 MAC (temac_block)

1.基本概念

TEMAC 是支持多速度模式的以太网媒体访问控制控制器，核心作用是实现数据链路层功能，管理以太网帧的收发、校验，并负责与物理层的通信适配。

2.主要功能

速度自适应：支持 10Mbps、100Mbps、1000Mbps 三种工作速度模式；

帧管理：完成以太网帧的封装、发送、接收及 CRC 校验；

接口转换：实现 GMII/RGMII 接口与客户端逻辑的数据格式转换；

时钟管理：生成并同步不同速度模式下的时钟信号；

流控支持：遵循 IEEE 802.3x 标准，实现以太网流量控制。

3.硬件实现特点

模块化设计：由时钟生成、RGMII 接口、MAC 核心等功能子模块组成；

双时钟域设计：发送与接收通道采用独立时钟域，需进行跨时钟域同步；

速度适应机制：通过时钟使能信号调整不同速度模式下的数据传输速率；

状态指示：通过 tri_speed 信号实时指示当前工作速度。

4.MAC 帧格式（遵循 IEEE 802.3 标准）

字段	字节数	功能描述
前导码 (Preamble)	7	实现接收端与发送端同步
帧起始定界符 (SFD)	1	标识以太网帧的开始
目的 MAC 地址	6	接收设备的物理地址

字段	字节数	功能描述
源 MAC 地址	6	发送设备的物理地址
长度 / 类型 字段	2	标识有效载荷长度(≤1500 字节时)或上层协议类型(如 IP、ARP 等)
有效载荷	46-1500	上层传输的实际数据, 不足 46 字节时会进行填充
帧校验序列 (FCS)	4	32 位 CRC 校验, 用于检测数据在传输过程中是否发生错误, 保障数据完整性

5.与物理层的接口方式 (RGMII)

TEMAC 通过 RGMII (Reduced Gigabit Media Independent Interface) 接口与物理层连接, 核心特性如下:

数据位宽: 4 位数据总线, 采用 DDR (Double Data Rate) 技术, 双沿采样实现 8 位数据传输;

时钟同步: 采用源同步时钟, 收发时钟由物理层提供或 MAC 生成;

控制信号: 通过专用控制信号指示数据有效性及错误状态;

速度适配: 不同速度模式下动态调整时钟频率与数据采样方式 (由 rgmii_interface.v 实现)。

四、UDP/IP 协议栈与 TEMAC 的交互机制

1.数据发送流程

应用层发起发送请求: 通过 app_tx_request 及相关信号将数据传输至 UDP/IP 协议栈;

协议栈封装: UDP/IP 协议栈按层次添加 UDP 头、IP 头、MAC 头, 形成以太网帧;

数据传输至 TEMAC: 通过 temac_tx_data、temac_tx_valid 等信号将帧数据传递给 TEMAC;

TEMAC 帧处理: TEMAC 添加前导码、SFD 字段, 完成 CRC 校验;

RGMII 接口转换: 将并行数据转换为 RGMII 接口格式;

物理层发送: 数据经 RGMII 接口传输至物理层芯片, 转换为电信号发送至网络。

2.数据接收流程

物理层接收: 物理层芯片接收网络电信号, 转换为 RGMII 接口信号;

RGMII 接口转换: 将 RGMII 格式数据还原为并行数据;

TEMAC 帧处理: TEMAC 接收数据并进行 CRC 校验, 筛选有效以太网帧;

数据传输至协议栈: 通过 temac_rx_data、temac_rx_valid 等信号将有效帧传递给 UDP/IP 协议栈;

协议栈解封: 逐层解析 MAC 头、IP 头、UDP 头, 提取应用层数据;

数据传输至应用层: 通过 app_rx_data、app_rx_data_valid 等信号将数据传递给上层应用。

3.时钟同步机制

发送时钟: 采用固定 125MHz 时钟, 保障高速传输稳定性;

接收时钟: 随网络速度动态变化 (1Gbps 时 125MHz、100Mbps 时 25MHz、10Mbps 时 2.5MHz) ;

适配机制：低速模式下通过时钟使能信号协调数据传输速率，确保跨时钟域数据同步。

4. 流量控制机制

系统采用多层流量控制，避免数据溢出：

应用层流控：通过 `udp2app_tx_ready` 与 `app_tx_ack` 信号调节应用数据发送节奏；

协议栈与 TEMAC 流控：通过 `temac_tx_ready` 信号控制协议栈向 TEMAC 的数据传输；

MAC 层流控：支持 IEEE 802.3x 标准，通过发送暂停帧调节对端发送速率。

五、总结

UDP/IP 协议栈与 TEMAC 以太网 MAC 通过模块化设计和标准化接口实现高效协同，构建了完整的网络通信链路：UDP/IP 协议栈负责网络层与传输层的协议处理，完成数据的封装与解封；TEMAC 负责数据链路层的帧管理与物理层接口适配，支持多速度模式和流量控制。二者的有机结合保障了网络数据传输的可靠性、灵活性和高效性，可根据实际应用场景优化配置，满足不同速率、不同场景下的网络通信需求。

2.2.7 子板控制模块

1. 本地同步显示功能实现分析

本地同步显示的核心目标是确保控制器本地界面与发送至主板的内容完全一致，其实现依赖共享数据源设计、实时性保障措施和一致性校验逻辑，具体如下：

2. 数据同步机制

通过“单一数据源 + 统一格式封装”实现数据一致性，避免多源头导致的偏差：

全局共享状态变量：在顶层模块 `top.v` 中维护核心状态变量，包括单色 LED 计数器 (`led_single`)、双色 LED 计数器 (`dual_counter`)、数码管位选 (`digit_pos`) 和数码管值数组 (`digit_val`)。这些变量同时连接至 UDP 发送模块和本地显示模块，确保两者获取完全相同的数据基础。

标准化数据打包：为保证与主板的数据兼容性，顶层模块将数码管数据按固定格式打包，具体 Verilog 实现如下：

```
verilog
wire [31:0] digit_values;
assign digit_values = {
    digit_val[7], digit_val[6], digit_val[5], digit_val[4],
    digit_val[3], digit_val[2], digit_val[1], digit_val[0]
};
```

统一的打包格式确保本地显示与发送至主板的数据结构完全一致。

3. 实时性保障措施

通过“时钟同步 + 直接数据交互”减少延迟，确保显示与传输的实时同

步：

统一时钟域设计：系统关键模块（UDP 发送模块、本地显示模块）均采用 `udp_clk` 时钟域，由 `udp_clk_gen` 模块提供稳定同步的时钟信号，避免跨时钟域传输导致的延迟和亚稳态问题。

无缓冲直接连接：全局状态变量直接接入本地显示模块（`local_led_display.v`）和 UDP 发送模块，无中间缓冲或异步处理环节，最大化降低数据传输延迟。

快速事件响应：按键处理采用边沿检测电路，确保按键操作被及时捕获；状态变量更新逻辑集中在顶层模块，减少模块间通信延迟，实现状态的快速刷新。

4.显示内容一致性校验方法

通过“单一控制 + 同时驱动”逻辑，无需显式校验即可保障一致性：

单一控制点：所有状态变量的更新集中在顶层模块的同一 `always` 块中，确保状态变更的原子性；本地显示与 UDP 发送共用同一组按键、拨码开关控制，避免独立控制导致的数据偏差。

双模块同时驱动：本地显示模块与 UDP 发送模块基于同一数据源同时工作，本地显示实时反映当前发送（或待发送）的数据，天然保证两者内容一致。

实现流程图如下：

（一）本地同步显示功能流程图

流程环节	具体描述
输入触发	按键/拨码开关作为输入设备，用户操作后产生信号
核心处理	顶层模块（系统控制核心）接收信号，更新全局状态变量（存储系统当前状态/参数）
本地反馈	驱动本地显示模块，通过 LED/数码管实时展示更新后的状态，供用户直观查看
远程传输	驱动 UDP 发送模块，将状态数据标准化打包后，经以太网以 UDP 协议发送至主板，实现远程数据上报/处理

（二）模式命令优先机制流程图

步骤序号	操作内容
1	拨码开关改变模式
2	顶层模块检测到模式变化（ <code>prev_mode != mode</code> ）
3	<code>udp_led_tx</code> 模块状态机切换
4	进入 <code>MODE_DELAY</code> （短暂延时）
5	进入 <code>SEND_MODE_CMD</code> （发送模式命令）
6	进入 <code>WAIT_MODE_ACK</code> （等待主板确认）
7	确认完成
8	进入 <code>SEND_DATA</code> （传输普通数据）
9	执行正常数据传输流程

技术实现关键点如下：

1.共享数据源：本地同步显示的核心，通过全局统一的状态变量，从根源上保证显示与传输内容的一致性。

2.状态机控制：模式命令优先级的核心实现方式，通过明确的状态定义和有序转移，解决命令与数据的传输冲突。

3.统一时钟域：避免跨时钟域延迟和亚稳态，保障数据处理和传输的实时性。

4.固定命令格式：简化模式命令的传输控制，提升命令识别的可靠性，降低冲突风险。

以上设计共同保障了系统在复杂操作场景下的稳定性，实现了本地显示与远程传输的同步性、模式命令的优先响应性。

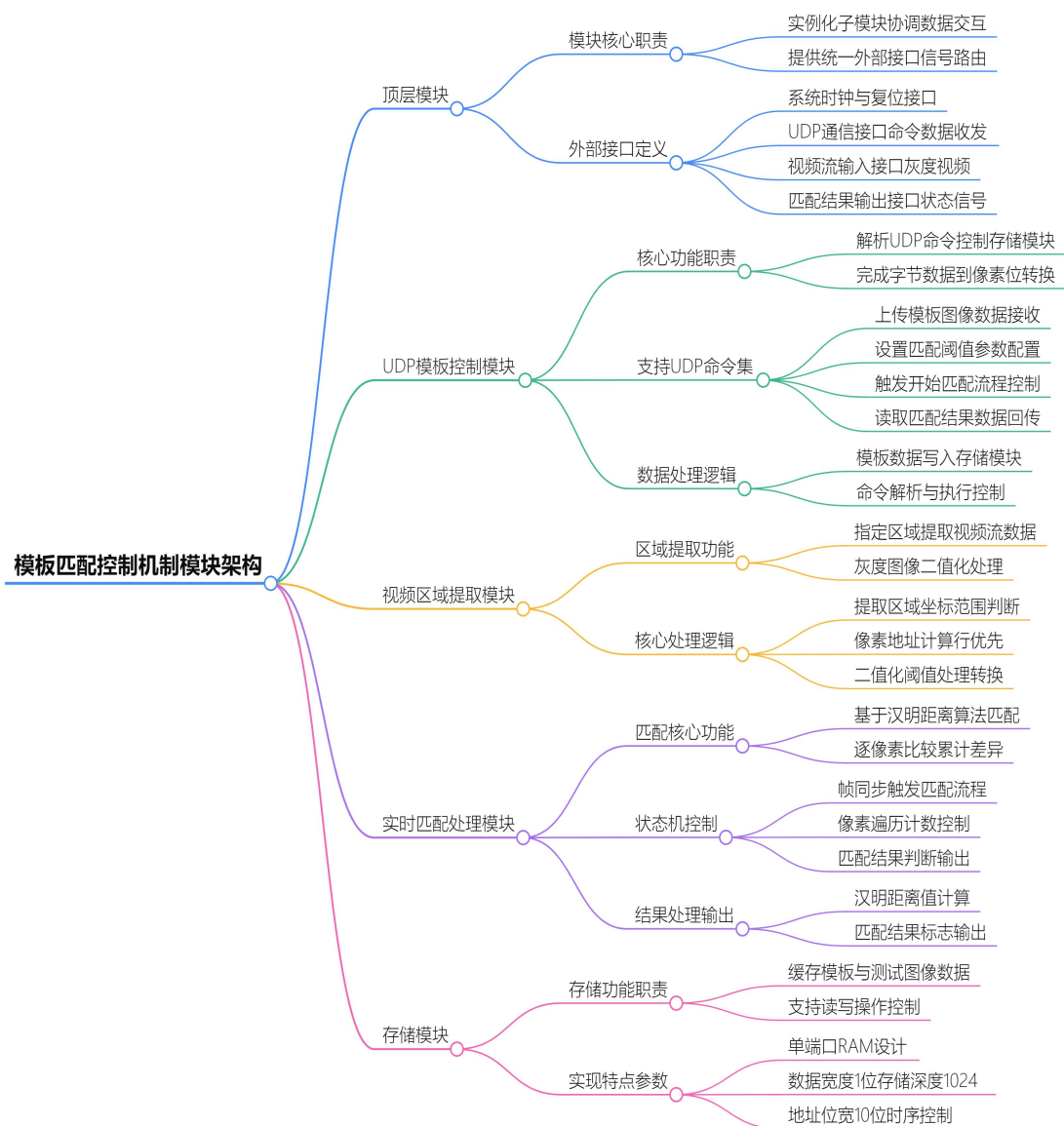
（二）具体图形算法设计

2.2.8 模板匹配控制机制模块

一、整体架构设计

模板匹配控制机制是基于 FPGA 实现的实时视频流模板匹配系统，采用模块化设计思想，通过多模块协同工作完成模板上传、区域提取、实时匹配及结果输出全流程。

系统核心架构包含 1 个顶层封装模块、4 个功能子模块及 1 个存储模块，各模块职责明确、接口标准化，整体架构如下：



核心组件说明

1.顶层模块（vip_template_match）：系统入口，实例化所有子模块，负责模块间信号路由与数据交互，提供统一外部接口；

2.UDP 模板控制模块（udp_template_ctrl）：处理 UDP 通信，解析命令、接收模板 / 测试图像、设置阈值，控制存储模块写入；

3.区域提取模块（template_region_extract）：从实时视频流中提取指定区域，完成灰度图二值化处理；

4.实时匹配模块（template_match_realtime）：核心算法模块，基于汉明距离实现模板与提取区域的逐像素匹配；

5.存储模块（myram）：参数化单端口 RAM，缓存模板图像和测试图像数据；

6.核心匹配算法模块（template_match）：通用匹配算法实现，提供三段式状态机控制的匹配逻辑。

基本流程表

步骤序号	步骤名称	具体描述
1	模板上传	通过 UDP 接口接收 32×32 像素的二值化模板图像（128 字节数据）。
2	区域提取	从左目摄像头实时视频流中，截取左上角 (0,0)~(31,31) 的 32×32 像素区域。
3	二值化处理	以 128 为阈值，将提取区域的灰度图像转换为 1 位二值图像（像素值>128 为 1，否则为 0）。
4	实时匹配	每帧触发一次匹配，逐像素比较模板与当前帧提取区域；
5	结果判断	累计不同像素数量（汉明距离），与设定阈值比较，输出“匹配 / 不匹配”结果。

二、关键算法：汉明距离匹配

汉明距离用于衡量两个等长二进制数据的差异程度，系统中通过异或运算实现像素差异检测，核心公式如下：

plaintext

// 汉明距离计算：累计异或结果为 1 的像素数（不同像素）

HammingDistance = $\sum(\text{pixel_template}[i][j] \text{ XOR } \text{pixel_test}[i][j])$ ($i,j \in [0,31]$)

// 匹配结果判断：汉明距离≤阈值则判定为匹配

MatchResult = (HammingDistance ≤ Threshold) ? 1 : 0

三、顶层模块：vip_template_match

1. 核心职责

整合所有子模块，协调 UDP 通信、视频处理、匹配计算的时序；

提供系统全局接口（时钟、复位、UDP、视频输入、结果输出）；

传递模块间关键信号（如模板数据、提取区域数据、匹配阈值、帧同步信号）。

外部接口定义如下：

接口类别	接口名称	方向	宽度	描述
系统时钟与复位接口	clk	input	1位	系统主时钟
系统时钟与复位接口	rst_n	input	1位	异步复位（低有效）
UDP 通信接口	udp_rx_valid	input	1位	UDP 接收数据有效
UDP 通信接口	udp_rx_cmd	input	15:0	UDP 接收命令
UDP 通信接口	udp_rx_data	input	7:0	UDP 接收数据
UDP 通信接口	udp_rx_byte_cnt_val	input	1位	接收字节计数有效
UDP 通信接口	udp_rx_byte_cnt	input	15:0	接收字节计数
UDP 通信接口	udp_tx_req	output	1位	UDP 发送请求
UDP 通信接口	udp_tx_data	output	7:0	UDP 发送数据
UDP 通信接口	udp_tx_done	input	1位	UDP 发送完成
视频流输入接口	video_hsyn	input	1位	视频行同步
视频流输入接口	video_vsyn	input	1位	视频场同步
视频流输入接口	video_en	input	1位	视频数据有效
视频流输入接口	video_gray	input	7:0	灰度视频数据
匹配结果输出接口	match_result	output	1位	匹配结果（1=匹配）
匹配结果输出接口	match_result_valid	output	1位	结果有效标志
匹配结果输出接口	hamming_distance	output	9:0	汉明距离值

四、UDP 模板控制模块：udp_template_ctrl

1. 核心职责

解析 UDP 命令，实现模板上传、测试图像上传、阈值设置、匹配触发、结果读取功能；

完成“字节数据→像素位”转换（128 字节→1024 位，32×32 像素）；

控制 myram 存储模块的读写，传递匹配控制信号。

支持的 UDP 命令如下：

命令代码	功能描述	数据长度	说明
0x00F0	上传模板图像	128 字节	32×32 二值图像，1 字节 = 8 像素（共 32×32=1024 像素，按位存储黑白信息）
0x00F1	上传测试图像	128 字节	同模板图像格式
0x00F2	设置匹配阈值	2 字节	汉明距离最大允许值（控制匹配精度，值越小要求越严格）
0x00F3	开始匹配	无	触发单帧图像匹配流程
0x00F4	读取匹配结果	无	返回 1 字节数据（包含匹配状态标志及实际计算的汉明距离）

2. 核心逻辑：模板数据写入

将 UDP 接收的 128 字节数据拆分为 1024 位像素（MSB 优先），写入 myram 存储模块：

```

verilog
always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        template_we    <= 1'b0;
        template_addr <= 10'd0;
        template_data <= 1'b0;
    end
    else if(cmd_valid && current_cmd == CMD_UPLOAD_TEMPLATE &&
udp_rx_byte_cnt_valid) begin
        // 跳过命令头，仅处理数据段（字节计数 2~130）
        if(udp_rx_byte_cnt >= 16'd2 && udp_rx_byte_cnt < (IMG_BYTES +
2)) begin
            template_we    <= 1'b1;           // 写使能
            template_addr <= pixel_cnt;       // 像素地址（0~1023）
            template_data <= udp_rx_data[7 - bit_cnt]; // 逐位提取字节数
            据
        end
    end
end
else

```

```

        template_we <= 1'b0;
    end
    else
        template_we <= 1'b0;
    end
    区域提取模块: template_region_extract
    3.提取区域判断 (左上角 32×32 像素)
    localparam REGION_X_START = 5'd0;    // 区域 X 起始坐标
    localparam REGION_Y_START = 5'd0;    // 区域 Y 起始坐标
    localparam REGION_WIDTH  = 5'd32;    // 区域宽度
    localparam REGION_HEIGHT = 5'd32;    // 区域高度
    wire in_region_x = (h_cnt >= REGION_X_START) && (h_cnt <
REGION_X_START + REGION_WIDTH);
    wire in_region_y = (v_cnt >= REGION_Y_START) && (v_cnt <
REGION_Y_START + REGION_HEIGHT);
    wire in_region = in_region_x && in_region_y && video_en; // 区域内且数
据有效
    4. 像素地址计算 (行优先: y*32 + x)
    wire [4:0] region_x = h_cnt[4:0]; // 区域内 X 坐标 (0~31)
    wire [4:0] region_y = v_cnt[4:0]; // 区域内 Y 坐标 (0~31)
    wire [9:0] pixel_addr = {region_y, region_x}; // 10 位地址 (0~1023)
    5. 二值化处理 (阈值 128)
    localparam BINARY_THRESHOLD = 8'd128;
    wire pixel_binary = (video_gray > BINARY_THRESHOLD); // 1=亮, 0=暗

```

五、实时匹配模块: template_match_realtime

1. 核心职责

基于帧同步信号触发每帧匹配;

逐像素读取模板数据与当前帧提取区域数据, 计算汉明距离;
与阈值比较输出匹配结果, 并通过 UDP 回传。

2. 核心逻辑: 匹配状态机

verilog

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        diff_count          <= 10'd0;    // 不同像素计数器
        pixel_count          <= 10'd0;    // 像素遍历计数器
        comparing            <= 1'b0;    // 比较状态标志
        match_result         <= 1'b0;    // 匹配结果
        match_result_valid <= 1'b0;    // 结果有效标志
        hamming_distance     <= 10'd0;    // 汉明距离
    end else begin
        match_result_valid <= 1'b0; // 默认清除有效标志

        if (frame_vsyn_rise) begin // 场同步上升沿, 开始新帧匹配

```

```

diff_count <= 10'd0;
pixel_count <= 10'd0;
comparing <= 1'b1;
end else if (comparing && in_region) begin // 遍历区域内所有像素
    pixel_count <= pixel_count + 1'b1;

    // 异或运算检测像素差异，累计计数
    if (pixel_template != pixel_binary)
        diff_count <= diff_count + 1'b1;

    // 遍历完所有像素（32×32=1024 个），输出结果
    if (pixel_count == (TOTAL_PIXELS - 1)) begin
        hamming_distance <= diff_count;
        match_result <= (diff_count <= match_threshold) ? 1'b1 :
1'b0;

        match_result_valid <= 1'b1;
        comparing <= 1'b0; // 结束本帧比较
    end
end
end
End

```

六、存储模块：myram

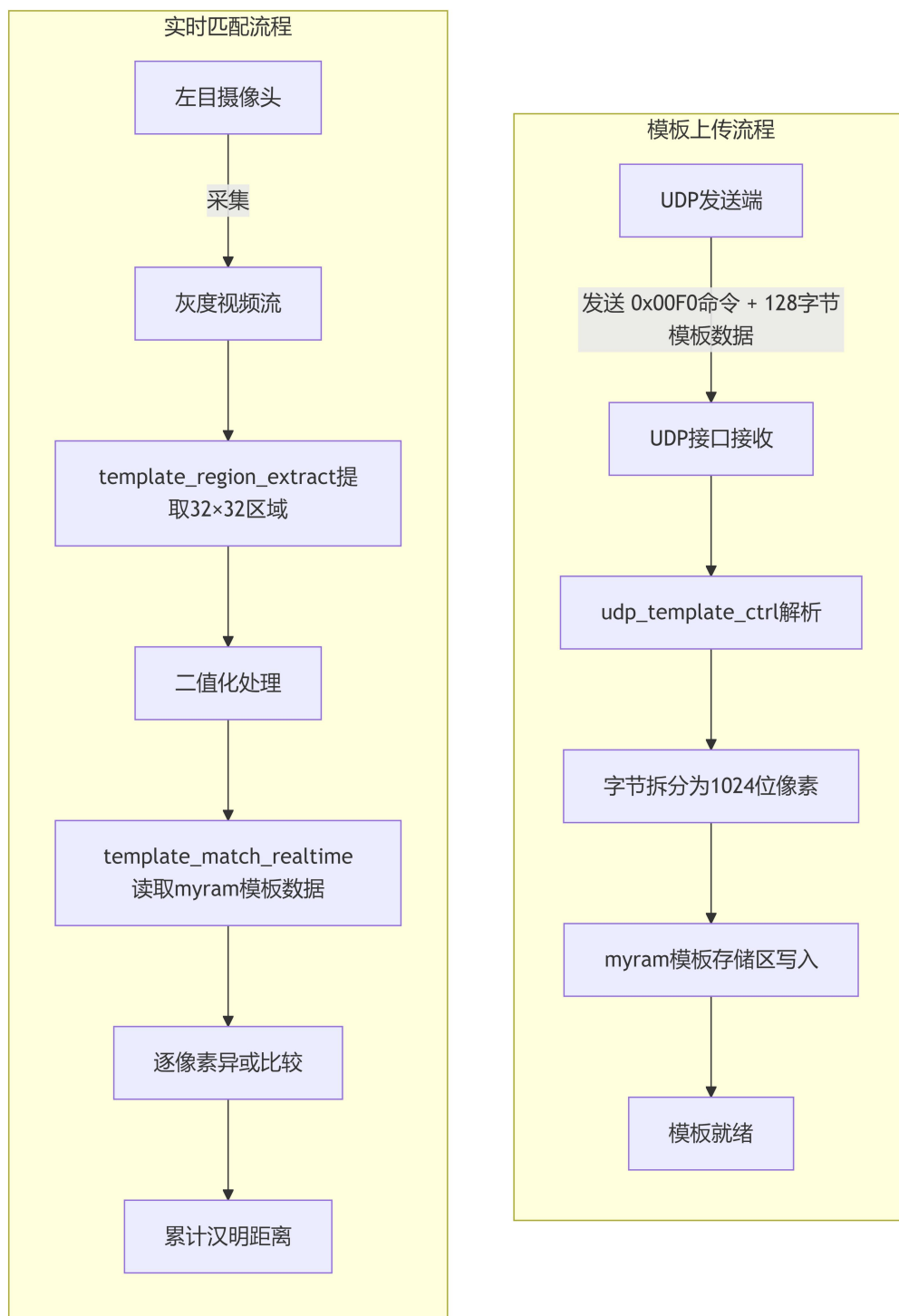
数据宽度：1 位（适配二值像素）。

存储深度：1024（32×32 像素）。

地址位宽：10 位。

单端口设计：读写共享地址总线，时序上保证“先读后写”或“先写后读”无冲突。

数据流程分析图如下：



关键参数与配置如下：

参 数 名 称	默认值	描述	配置方式
模板/提取区域尺寸	32 × 32 像素	匹配的图像尺寸	代码参数化 (REGION_*)
二值化阈值	128	灰度转二值的判定标准	代 码 参 数 化 (BINARY_THRESHOLD)
匹 配 阈 值	50	判定匹配的最大汉明距离	UDP 命令 0x00F2 动态设置
输入视频分辨率	640×480	左目摄像头视频分辨率	系统全局配置
存储模块深度	1024 (10 位地址)	缓存像素数量	-

模块间交互方式如下：

(1) 顶层模块与子模块交互

与 udp_template_ctrl: 传递 UDP 接收的命令 / 数据, 接收匹配结果并触发 UDP 回传;

与 template_region_extract: 提供视频流 (hsyn/vsyn/en/gray), 接收提取区域的像素地址、二值数据;

与 template_match_realtime: 传递匹配阈值、模板数据, 接收汉明距离和匹配结果。

(2) 子模块间直接交互

udp_template_ctrl ↔ myram: 通过“写使能 + 地址 + 数据”信号写入模板 / 测试图像;

template_region_extract ↔ template_match_realtime: 传递提取区域的像素数据、地址及有效标志;

template_match_realtime ↔ myram: 通过地址读取模板数据, 用于匹配计算。

七、总结

本模板匹配控制机制是一款高度集成的 FPGA 实时视频处理系统, 核心优势如下:

模块化设计: 各模块职责独立、接口标准化, 便于维护与扩展; 实时性强: 基于帧同步触发匹配, 每帧处理延迟仅 1024 个时钟周期, 适配 640×480 分辨率的实时视频流; 灵活性高: 支持 UDP 动态配置模板、测试图像和匹配阈值, 无需修改硬件代码; 资源消耗低: 采用二值化处理和汉明距离算法, 避免复杂运算, 适合 FPGA 硬件实现。系统适用于实时视觉检测、目标定位等场景, 通过优化提取区域灵活性 (支持动态配置区域坐标)、增加多模板匹配功能, 可进一步扩展应用范围。

2.2.9 帧差法运动检测模块

一、帧差法数学原理与硬件实现

(1) 数学原理

帧差法的基本数学表达式为： $D(x,y) = |I(t,x,y) - I(t-1,x,y)|$ 其中：

$D(x,y)$ 表示坐标 (x,y) 处的帧差绝对值

$I(t,x,y)$ 表示当前帧在 (x,y) 位置的像素值

$I(t-1,x,y)$ 表示前一帧在 (x,y) 位置的像素值

(2) . 硬件实现代码

在 `frame_diff_downsample.v` 中，帧差计算通过有符号数减法和符号位判断实现绝对值计算，确保差值始终为正值：
`wire signed [8:0] diff_signed = $signed({1'b0, i_gray}) - $signed({1'b0, prev_gray});`
`wire [7:0] diff_abs = diff_signed[8] ? (~diff_signed[7:0] + 1'b1) : diff_signed[7:0];`

二、Downsample（下采样）的作用与实现方式

(1) 下采样的作用

降低计算复杂度：通过减少像素数量，显著降低存储和计算资源需求

减少噪声干扰：下采样过程相当于一种低通滤波，可以平滑图像，减少随机噪声的影响

提高实时性：处理更少的像素点，使算法能够在有限硬件资源下实现实时处理

扩大检测范围：降低分辨率可以更好地捕捉整体运动，减少对细微变化的过度敏感

(2) 实现方式

本模块采用 4×4 降采样策略，具体实现如下：
 // 降采样判断（每 4×4 取左上角）
`wire is_sample_point = (h_cnt [1:0] == 2'b00) && (v_cnt [1:0] == 2'b00) && i_en;`
 // 降采样地址（ 160×120 ）
`wire [14:0] ds_addr = {v_cnt [8:2], h_cnt [9:2]};`
 // 7 位 $v + 8$ 位 $h = 15$ 位。

该方法的特点：

每 4×4 像素块中仅保留左上角的像素值

将 640×480 分辨率的图像降采样至 160×120

采样点选择使用像素坐标的低位判断，实现简单高效

降采样后使用 BRAM 存储，仅需 $19200 \times 8\text{bit}$ 的存储空间（约 5 个 BRAM 32k）

帧差法运动检测的完整算法流程

(1) 图像预处理

灰度转换：输入为灰度图像（8bit），已由上游模块 `image_proc_wrapper.v` 处理完成

像素位置计数：通过行计数器 `h_cnt` 和列计数器 `v_cnt` 跟踪当前处理的像素位置

// 像素位置计数器（ 640×480 ）

`reg [9:0] h_cnt; // 0-639`

`reg [8:0] v_cnt; // 0-479`

(2) 帧差计算

前一帧存储：使用 BRAM 存储前一帧的下采样结果

实时帧差：计算当前帧与前一帧相同位置像素的差值绝对值

差值增强：对差值进行放大处理，提高运动检测的灵敏度

// 差值增强 (×2, 增强对比度)

```
wire [8:0] diff_enhanced = {diff_abs, 1'b0}; // 左移 1 位 = ×2
```

```
wire [7:0] diff_final = (diff_enhanced > 9'd255) ? 8'd255 : diff_enhanced [7:0];
```

(3) 阈值分割

设置固定阈值, 将差值图像二值化, 判断是否存在运动: // 运动检测阈值 (可调整) parameter MOTION_THRESHOLD = 8'd30; // 差值 > 30 认为有运动 wire is_motion = (diff_final > MOTION_THRESHOLD);

(4) 噪声去除与后处理

采样点保持: 使用寄存器保持采样点的差值和运动标志, 直到下一个采样点

最近邻插值: 对于非采样点位置, 重用最近采样点的运动检测结果

// 差值寄存器和运动标志寄存器 (保持 4×4 周期)

```
reg [7:0] diff_hold;
```

```
reg motion_hold;
```

```
always @(posedge i_clk or negedge i_rst_n) begin
```

```
if (!i_rst_n) begin
```

```
diff_hold <= 8'd0;
```

```
motion_hold <= 1'b0;
```

```
end else if (is_sample_point_d1) begin // 采样点更新 (延迟后)
```

```
diff_hold <= diff_final;
```

```
motion_hold <= is_motion;
```

```
end
```

```
End
```

(5) 运动区域标记

在 motion_border_overlay.v 模块中, 对检测到的运动区域进行边框标记:

边缘检测: 检测运动区域的边缘 (当前像素有运动, 而相邻像素无运动)

边框绘制: 在边缘位置叠加绿色边框, 直观显示运动区域

// 边缘检测 (简化版: 检测水平和垂直边缘)

```
wire is_border = i_motion && (
```

```
// 左边缘: 当前有运动, 左侧无运动
```

```
(motion_shift_reg [2] == 1'b0) ||
```

```
// 右边缘: 当前有运动, 右侧无运动 (预测)
```

```
(motion_shift_reg [0] == 1'b0) ||
```

```
// 上边缘: 当前有运动, 上方无运动 (简化, 使用延迟)
```

```
(motion_prev_line == 1'b0)
```

```
);
```

三、关键参数设置与调整方法

(1) 下采样率

当前设置: 4×4 降采样 (640×480 → 160×120) 调整依据:

硬件资源限制: 降采样率直接影响 BRAM 使用量

检测精度需求: 更高的降采样率会降低空间分辨率, 但减少资源消耗

运动速度: 快速运动可能需要更高的降采样率以避免运动模糊

调整方法: 修改下采样判断逻辑中的位宽设置

// 修改位宽可以调整降采样率, 例如改为 3×3 降采样

```
wire is_sample_point = (h_cnt [1:0] == 2'b00) && (v_cnt [1:0] == 2'b00) &&
i_en;
```

(2) 差分阈值

当前设置: MOTION_THRESHOLD = 8'd30 调整依据:

场景光照条件: 高光照变化场景需要更高的阈值

运动灵敏度要求: 降低阈值会提高灵敏度, 但增加误检率

噪声水平: 高噪声环境需要较高的阈值来抑制噪声

调整方法: 修改参数定义, 可考虑实现为可配置输入

parameter MOTION_THRESHOLD = 8'd30; // 根据实际场景调整

(3) 噪声过滤窗口大小

当前实现: 隐式的 4×4 窗口 (通过降采样实现) 调整依据:

噪声特性: 不同类型的噪声需要不同大小的过滤窗口

运动细节保留: 较小的窗口保留更多细节, 但噪声过滤效果较差

调整方法: 可以在现有框架上添加显式的中值滤波或均值滤波模块

(4) 差值增强系数

当前设置: ×2 增强 (左移 1 位) 调整依据:

运动幅度: 低对比度场景可能需要更强的增强

显示效果: 增强系数影响最终视觉效果

调整方法: 修改差值增强逻辑

// 可调整为不同的增强系数

```
wire [8:0] diff_enhanced = {diff_abs, 1'b0}; // ×2
```

```
// 或 wire [9:0] diff_enhanced = {diff_abs, 2'b00}; // ×4
```

五. 总结

帧差法运动检测是一种简单高效的运动检测技术, 特别适合在资源受限的硬件平台上实现。本模块通过 4×4 降采样和 BRAM 存储优化, 在保证基本检测功能的同时, 有效降低了资源消耗。虽然存在对光照变化敏感、固定阈值等局限性, 但通过适当的参数调整和算法优化, 可以在许多实际应用场景中取得良好的效果。

2.2.10 双目立体视觉模块

一. 工作原理

(1) 基本原理

双目立体视觉系统模拟人类双眼感知深度的原理, 通过计算同一物体在左右两个摄像头成像中的位置差异 (视差) 来获取物体的深度信息[1]。

(2) 视差形成机制

当同一场景被两个水平放置的摄像头从不同角度拍摄时, 由于观察点差异, 同一物体在左右图像中的水平位置会产生偏移, 这个偏移量称为视差 (Disparity)。物体距离摄像头越近, 视差越大; 距离越远, 视差越小。

(3) 三维坐标计算几何关系

根据三角测量原理, 系统使用以下公式计算距离: $Distance = (Baseline \times Focal_Length) / Disparity$ 其中:

Distance: 物体到摄像头的距离

Baseline: 两个摄像头之间的距离（基线长度）

Focal_Length: 摄像头焦距（像素单位）

Disparity: 左右图像中同一物体的水平位置差

项目中通过 stereo_distance_calc.v 模块完成这一计算过程。

二. 核心组成部分

(1)硬件组件

双目摄像头: 系统使用两个相同规格的 OV5640 摄像头（左目和右目）。

FPGA 处理平台: 用于实时图像采集和处理。

时钟同步电路: 确保左右摄像头数据在同一时钟域下处理。

存储单元: 异步 FIFO 用于缓存右目数据，实现跨时钟域传输。

(2)软件模块

数据同步模块: 实现左右摄像头数据的时间对齐。

图像处理模块: 包括颜色空间转换、二值化处理等。

特征提取模块: 边界框检测，用于定位目标。

视差计算模块: 根据边界框位置计算视差。

距离转换模块: 将视差转换为实际距离。

数据传输模块: 将处理结果和图像数据通过 UDP 发送。

三. 关键技术指标

(1)物理参数

基线长度 (Baseline): 120mm (在 stereo_distance_calc 模块中可配置)

焦距 (Focal_Length): 350 像素 (根据摄像头特性设定)

(2)测量距离范围:

最小距离: 约 65.6cm (当视差最大为 640 像素时)

最大距离: 约 255cm (系统限制输出范围)

视场角: 由 OV5640 摄像头决定, 约 60-70 度

图像分辨率: 640×480 像素

(3)系统参数

处理帧率: 与摄像头帧率一致, 通常为 30fps

延迟: 流水线处理, 每帧约 3 个时钟周期的延迟

功耗: FPGA 实现的低功耗设计

数据接口:

摄像头输入: RGB565 格式

距离输出: 8 位 (0-255cm)

网络传输: UDP 协议

四. 子模块详细解析

1. dual_cam_delay - 双目数据同步与处理顶层模块

功能: 作为整个双目立体视觉系统的顶层模块, 负责同步左右摄像头数据并控制处理流程。

2.核心实现:

(1) 左目数据处理: 直接处理并打拍对齐

- (2) 右目数据同步：通过异步 FIFO 将右目数据同步到左目时钟域
- (3) 模式切换：支持 RGB 模式和测距模式的切换
- (4) 数据合并：将左右目数据合并为 32 位输出

```
assign cam_write_data = stereo_ranging_enable ? {left_binary_out,
left_binary_out, right_binary_out, right_binary_out} : // 测距模式：2 字节灰度
{cmos_wr_data_l_d1, cmos_wr_data_r_sync}; // RGB 模式：原始数据
```

3. 核心算法：

- (1) 像素坐标计数：跟踪当前处理的像素坐标
- (2) 极值检测：找出目标区域的最小 / 最大 X/Y 坐标
- (3) 中心计算：根据边界框极值计算中心点坐标

```
assign coor_data = x_max [10:1] + x_min [10:1]; // 等价于 (x_max + x_min) / 2
```

4. 模块特点：

- (1) 每帧更新一次边界框信息
- (2) 支持边界框可视化（在原图上绘制红色边框）
- (3) 输出目标中心点坐标用于视差计算

五、stereo_distance_calc - 距离计算模块

1. 模块功能：根据左右目边界框中心的视差计算物体距离。

2. 流水线架构：

- (1) 阶段 1：计算视差（左目 X 坐标 - 右目 X 坐标）
- (2) 阶段 2：根据公式计算距离（mm）
- (3) 阶段 3：转换为厘米并限制输出范围

```
always @(posedge clk or negedge rst_n) begin if (!rst_n) begin disparity_d1 <=
12'd0; distance_mm <= 32'd0; calc_stage2 <= 1'b0; end else if (calc_stage1)
begin disparity_d1 <= disparity; // 距离计算：D (mm) = (BASELINE *
FOCAL_LENGTH) / disparity if (disparity > 12'sd16 && disparity < 12'sd640)
begin distance_mm <= BASELINE_FOCAL / disparity; calc_stage2 <= 1'b1; end else
begin distance_mm <= 32'd0; calc_stage2 <= 1'b0; end end else begin calc_stage2 <=
1'b0; end end
```

3. 关键设计：

- (1) 视差范围限制：只处理有效视差范围（16-640 像素）
- (2) 距离转换优化：使用乘法近似除以 10（提高速度）
- (3) 参数化配置：基线长度和焦距可通过参数配置

六. 图像预处理优化

- (1) .颜色空间转换：高效的 RGB565 到 RGB888 和灰度转换。
- (2) .二值化处理：基于颜色阈值的目标提取，适用于特定颜色目标的检测。
- (3) .图像同步：采用异步 FIFO 确保左右图像的时间同步。

七. 立体匹配策略

项目采用了基于目标检测的简化匹配策略：

- (1) 目标检测：通过颜色二值化和边界框检测定位目标。
- (2) 中心点匹配：假设左右图像中相同颜色的目标是同一物体。

(3) 直接视差计算：直接比较左右目标中心点的 X 坐标差。

八. 总结

该双目立体视觉模块采用 FPGA 实现，通过视差原理计算目标距离，具有以下特点：

- (1) 实时性：硬件实现确保 30fps 的实时处理能力
- (2) 低功耗：FPGA 实现的低功耗设计
- (3) 简洁高效：采用简化的匹配策略，适合特定应用场景
- (4) 可扩展性：模块化设计，易于扩展和优化
- (5) 实用性：支持通过 UDP 接口远程配置和接收数据

虽然系统采用了一些简化假设如固定内参外参、基于颜色的目标检测，但其实现了完整的双目测距功能，可应用于多种实际场景。通过进一步优化和扩展，可以提高系统的精度、鲁棒性和适用范围。

2.3 软件系统介绍

2.3.1 软件整体介绍

1. 总体概述

prj.py 是一个基于 PyQt5 开发的 FPGA 综合控制系统上位机程序，实现了对 FPGA 开发板的远程控制与数据交互功能。该程序整合了 LED / 数码管控制、图像发送、视频接收、图像处理等多个功能模块，通过 UDP 协议与 FPGA 进行通信，实现实时控制与数据传输。

2. 系统整体架构

系统采用模块化设计，主要包含以下核心组件：

(1) 主应用程序类 (FPGAControlApp)：作为整个系统的核心控制器，负责整合各功能模块、管理界面交互和数据流转

(2) UDP 通信模块：实现与 FPGA 的数据收发功能

视频接收模块 (UDPCameraReceiver)：独立线程接收视频流数据

(3) UI 组件模块：自定义 UI 控件如 LEDWidget、DigitSelector 等

1. 功能模块详细划分

模块名称	主要功能	对应代码区域	交互方式
LED 控制模块	控制 4 个单色 LED 和 8 个双色 LED	create_led_page(), send_led_data()	发送 3 字节数据（掩码控制）
数码管控制模块	控制 8 位数码管和 4 个单色 LED	create_segment_page(), send_segment_data()	发送 8 字节数据
图片发送模块	发送不同分辨率的 RGB 图片	create_image_page(), send_image()	分包发送像素数据
视频接收模块	接收并显示实时视频流	create_video_page(), process_video_packets()	数据包解析与帧重组
网络通信模块	UDP 数据收发	toggle_connection(), send_mode_command()	UDP 协议通信

2.3.2 软件各模块介绍

一. 各模块介绍

1.网络通信模块

功能说明：负责与 FPGA 建立 UDP 连接，发送控制命令和接收数据。

核心实现：

```
python
运行
# 网络配置常量定义
DEFAULT_FPGA_IP = "192.168.240.1"
DEFAULT_LOCAL_IP = "192.168.240.2"
DEFAULT_CONTROL_PORT = 4
DEFAULT_VIDEO_PORT = 6001

# 连接控制函数
def toggle_connection(self):
    # 建立或断开连接
    # 验证 IP 格式和端口号
    # 启用/禁用网络配置界面
```


更新连接状态

关键技术点：

- (1) 使用 Python socket 库实现 UDP 通信
- (2) IP 地址格式验证确保连接安全性
- (3) 连接状态管理确保命令发送的有效性

2.LED 控制模块

功能说明：控制 FPGA 上的 4 个单色 LED 和 8 个双色 LED 的开关状态。

核心实现：

python

运行

```
def send_led_data(self):
    # 构建单色 LED 掩码（高 4 位有效）
    # 构建红色和绿色 LED 掩码（低电平有效）
    # 打包 3 字节数据并发送
    # 错误处理与状态反馈
```

数据格式：

byte0: 单色 LED 状态（高 4 位）

byte1: 绿色 LED 状态掩码（低电平有效）

byte2: 红色 LED 状态掩码（低电平有效）

3.图片发送模块

功能说明：选择本地图片，调整分辨率后发送到 FPGA 进行显示。

核心实现：

python

运行

分辨率配置

```
RESOLUTION_CONFIGS = {
    '640x480': {'width': 640, 'height': 480, 'cmd': 0x0030, 'display': '640×480
(VGA)', 'pixels': 307200},
    '1280x720': {'width': 1280, 'height': 720, 'cmd': 0x0031, 'display': '1280×720
(720p)', 'pixels': 921600},
    '1920x1080': {'width': 1920, 'height': 1080, 'cmd': 0x0032, 'display':
'1920×1080 (1080p)', 'pixels': 2073600}
}
```

```
def send_image(self):
```

- # 调整图片尺寸到目标分辨率
- # 水平翻转图像
- # 分包发送像素数据（每包 140 字节）
- # 实时更新发送进度

关键技术点：

使用 PIL 库进行图像处理（调整尺寸、水平翻转）

支持智能分辨率推荐算法
分包传输大数据量，避免网络拥塞
实时进度显示提升用户体验

4. 视频接收模块

功能说明：接收 FPGA 发送的实时视频流，解析数据包并重组为完整帧进行显示。

核心实现：

python

运行

```
class UDPCameraReceiver:
```

```
    # 后台线程接收 UDP 数据包
```

```
    def _receive_loop(self):
```

```
        # 循环接收数据包
```

```
        # 处理超时异常
```

```
        # 将接收到的数据放入队列
```

```
def process_video_packets(self):
```

```
    # 处理队列中的数据包
```

```
    # 检查帧超时
```

```
    # 维护 OpenCV 窗口响应
```

```
def handle_video_packet(self, packet):
```

```
    # 解析数据包头部（魔数、帧序号、包序号等）
```

```
    # 处理模板匹配结果
```

```
    # 重组完整视频帧
```

```
    # 计算 FPS 和带宽
```

数据格式：

数据包头部 32 字节，包含魔数 (0xAA0055FF)、帧序号、包序号等信息

数据包体 636 字节，包含图像数据

每帧由多个数据包组成，通过序号重组

二. 关键算法与数据处理流程

1. 视频帧重组算法

功能：将多个 UDP 数据包重组为完整的视频帧

算法流程：

（1）从队列中获取数据包

（2）调用 `handle_video_packet` 处理单个数据包

（3）检查帧超时情况

（4）维护 OpenCV 窗口响应

数据结构：

python

运行

使用字典存储各帧的数据包

```
self.frame_packets = {}
```

格式: {frame_seq: {packet_seq: packet_data, ...}, ...}

2..分辨率智能推荐算法

功能: 根据图片尺寸自动推荐最合适的显示分辨率

算法流程:

python

运行

```
def recommend_resolution(self, img_width, img_height):
```

```
    pixels = img_width * img_height
```

```
    # 根据像素数量判断推荐分辨率
```

```
    if pixels < 307200: # 小于 VGA
```

```
        return '640x480'
```

```
    elif pixels < 921600: # VGA 到 720p 之间
```

```
        return '1280x720'
```

```
    else: # 720p 及以上
```

```
        return '1920x1080'
```

3. 帧率与带宽计算算法

功能: 实时计算视频流的帧率和带宽

算法流程:

(1) 记录接收到的字节数和时间戳

(2) 统计完整帧数量

(3) 每秒钟计算:

帧率 = 帧数 / 时间间隔

带宽 = 字节数 / 时间间隔 / 1024 / 1024 * 8 (Mbps)

三. 核心函数分析与流程图

1.初始化函数 (init_ui)

功能: 初始化应用程序界面, 创建各功能模块的 UI 组件

输入参数: 无

输出: 无

流程:

(1) 创建菜单栏

(2) 创建网络配置区域

(3) 创建堆叠式功能页面 (LED、数码管、图片、视频、双目)

(4) 创建状态栏

(5) 设置初始页面为 LED 模式

(6) 初始化定时器

2. 模式切换函数 (switch_mode)

功能：切换不同的控制模式，发送对应命令到 FPGA

输入参数：

page_index: 页面索引

cmd_value: 命令值

mode_name: 模式名称

输出：无

流程：

- (1) 停止当前视频接收（如果正在接收）
- (2) 设置模式标志
- (3) 切换页面显示
- (4) 发送模式切换命令
- (5) 更新状态显示

3. 视频数据包处理函数 (process_video_packets)

功能：处理接收到的视频数据包，重组帧并显示

输入参数：无

输出：无

流程：

- (1) 从队列中获取数据包
- (2) 调用 handle_video_packet 处理单个数据包
- (3) 检查帧超时情况
- (4) 维护 OpenCV 窗口响应

4. 图像处理模式切换函数 (_send_camera_command)

功能：发送摄像头选择和图像处理模式命令

输入参数：

camera: 摄像头选择 ('left'/'right')

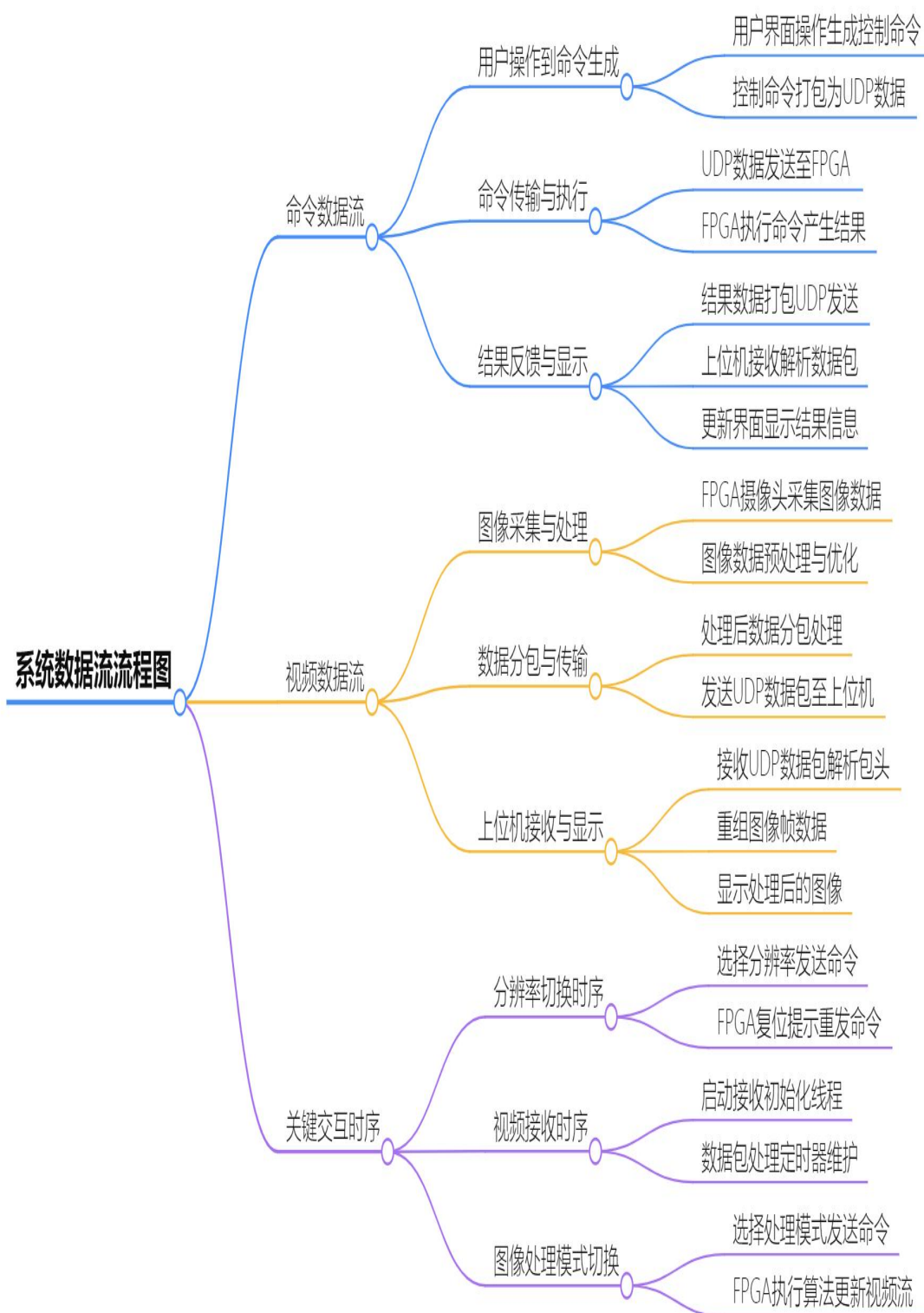
proc_mode: 处理模式 (0x00-0x07)

输出：无

流程：

- (1) 根据 camera 和 proc_mode 查找对应的命令值
- (2) 打包命令为 2 字节数据
- (3) 发送 UDP 数据包
- (4) 更新状态显示和日志

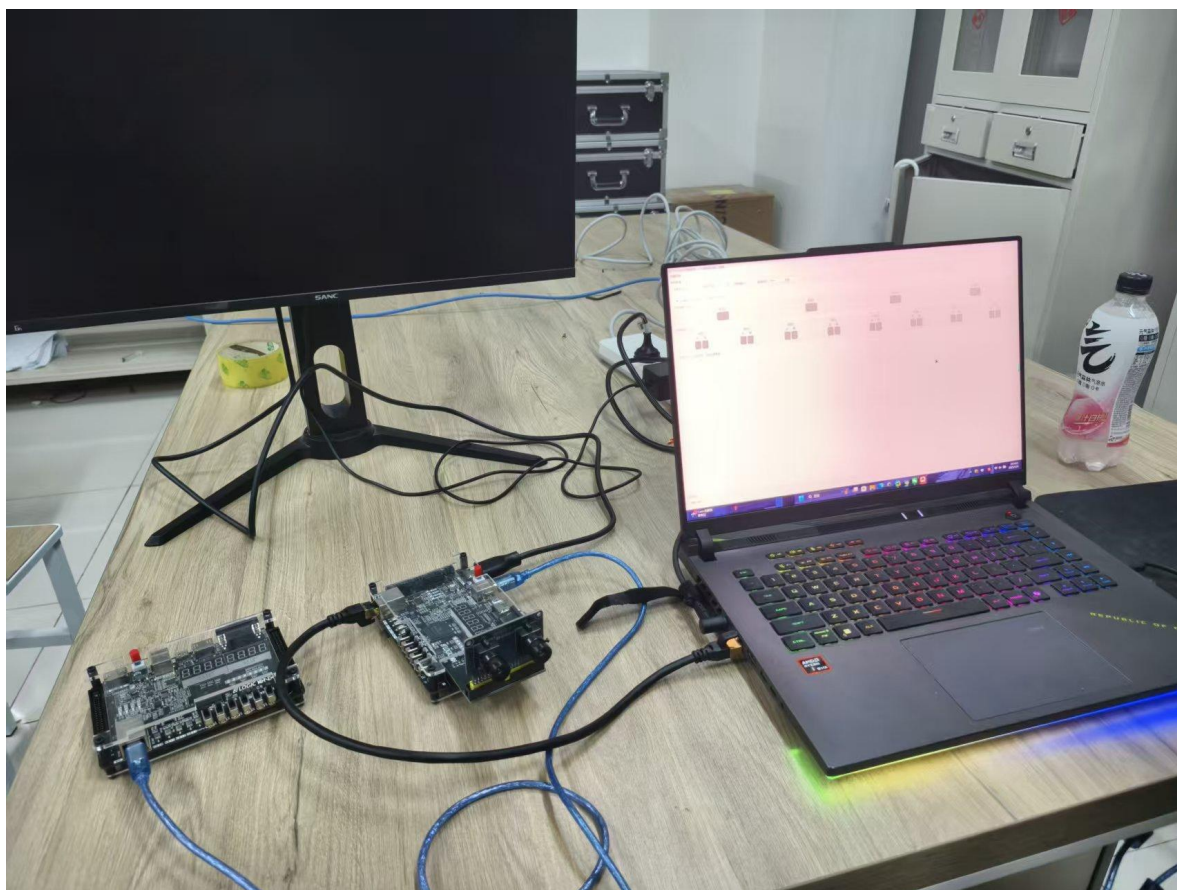
三. 系统交互与数据流向



2.3.2 系统数据流流程图

第三部分 完成情况及性能参数

3.1 整体介绍



3.1.1 系统全貌图



3.1.2 HX4S20C 开发板（正面）



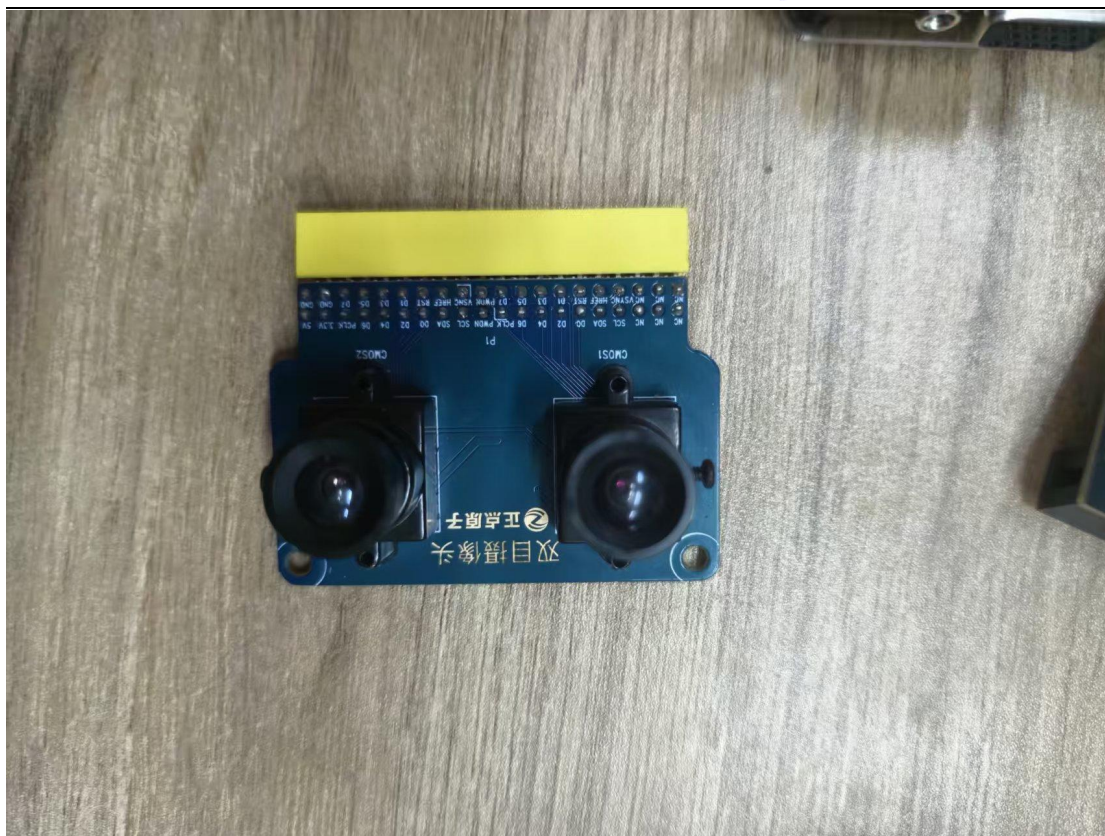
3.1.3 HX4S20C 开发板板（反面）



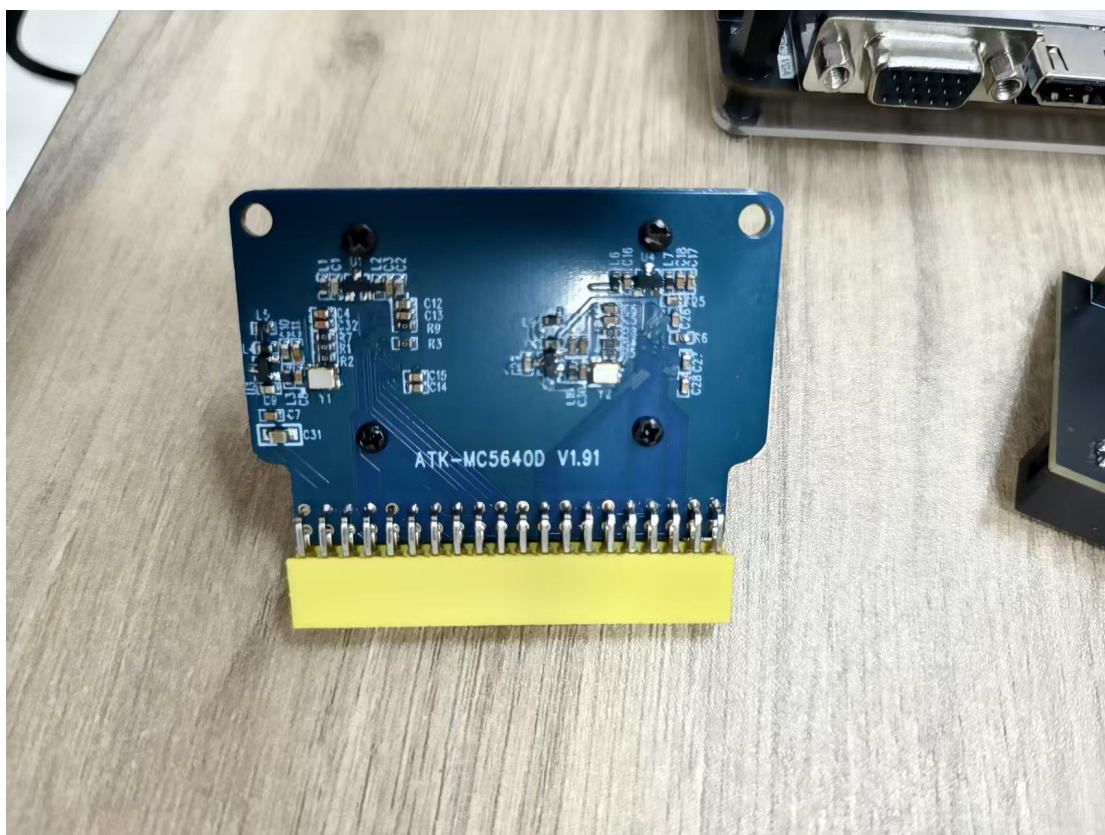
3.1.4 摄像头扩展板（正面）



3.1.5 摄像头扩展板（反面）

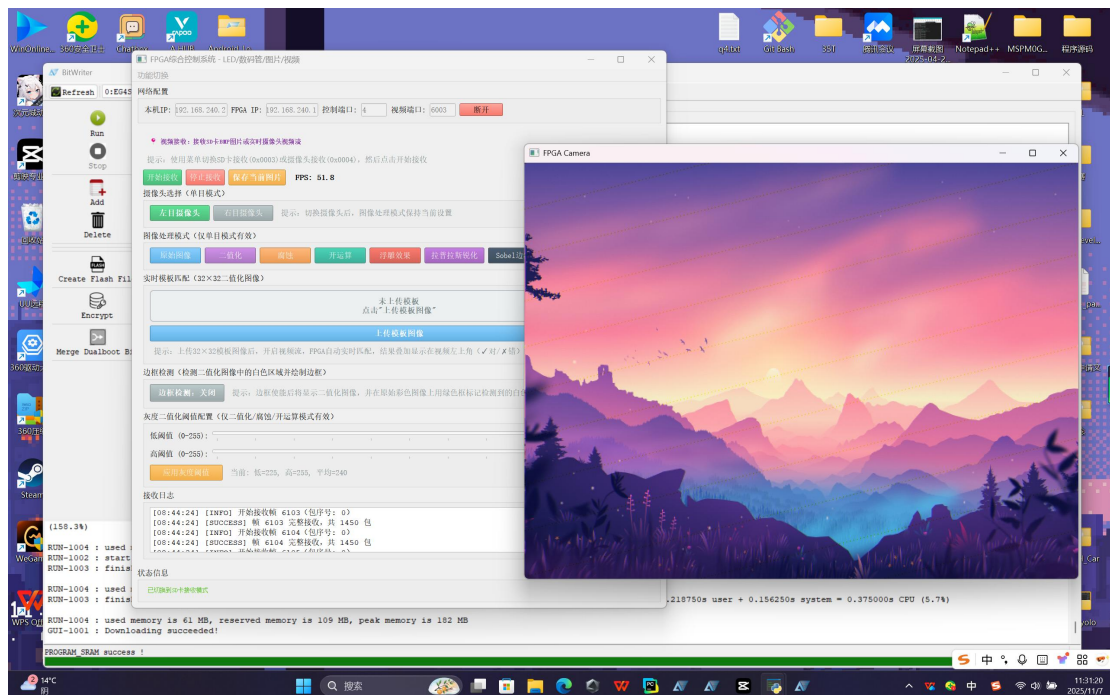


3.1.6 ov5640 双目摄像头（正面）

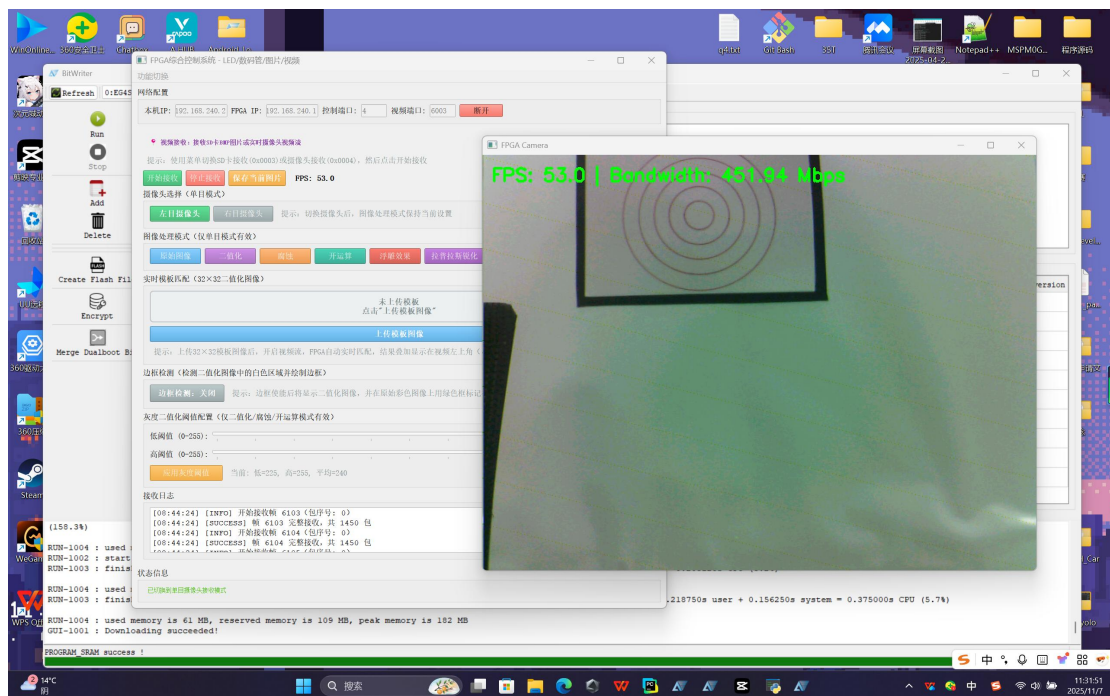


3.1.7 ov5640 双目摄像头（反面）

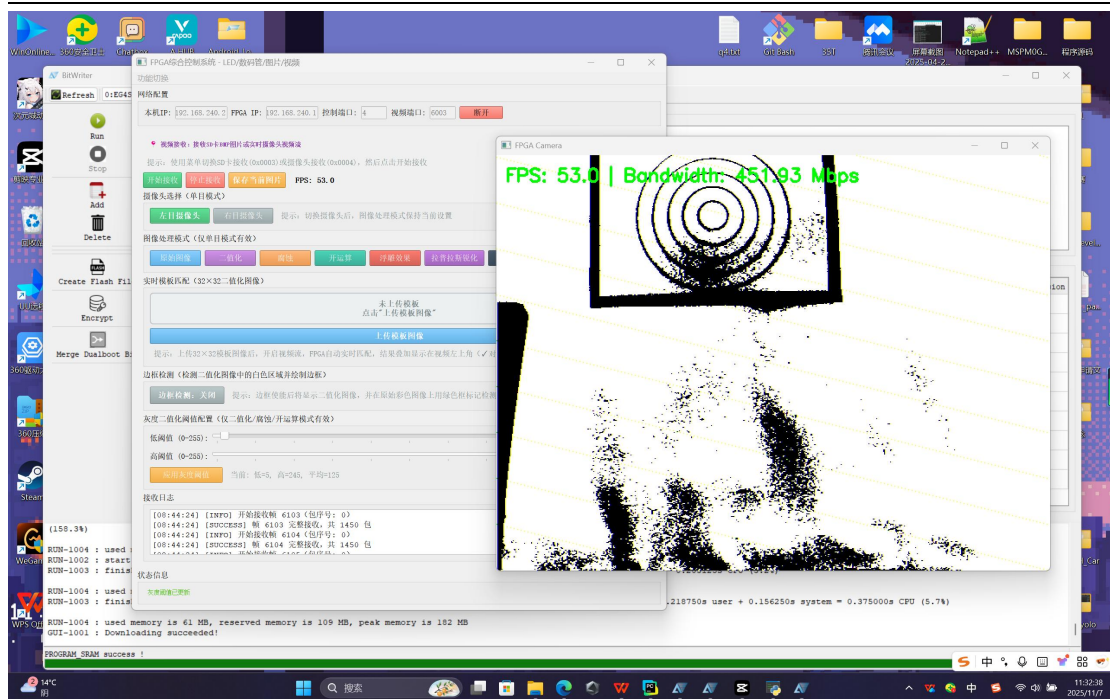
3.2 工程成果



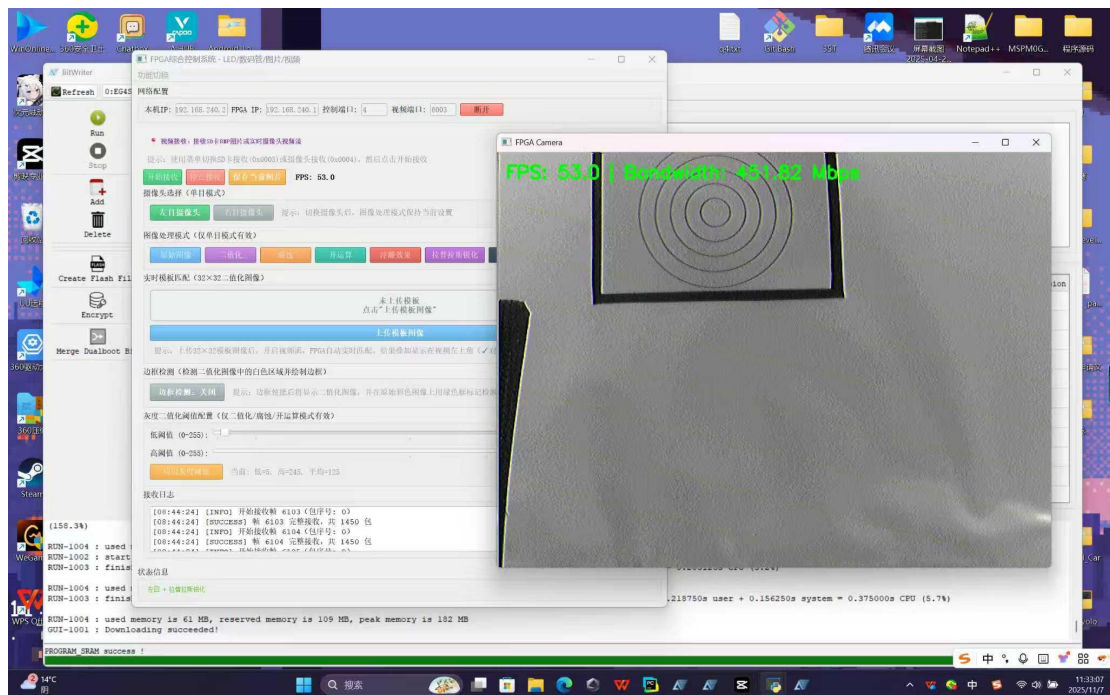
3.2.1 从 SD 卡中读取图像



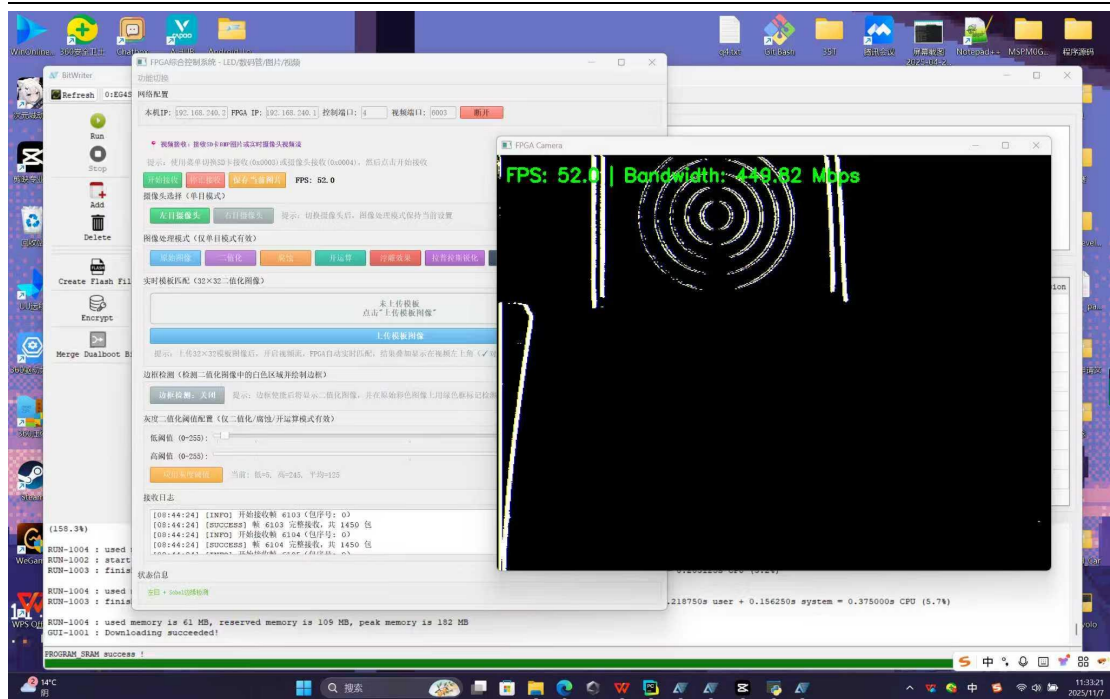
3.2.2 上位机接收 FPGA 端图像



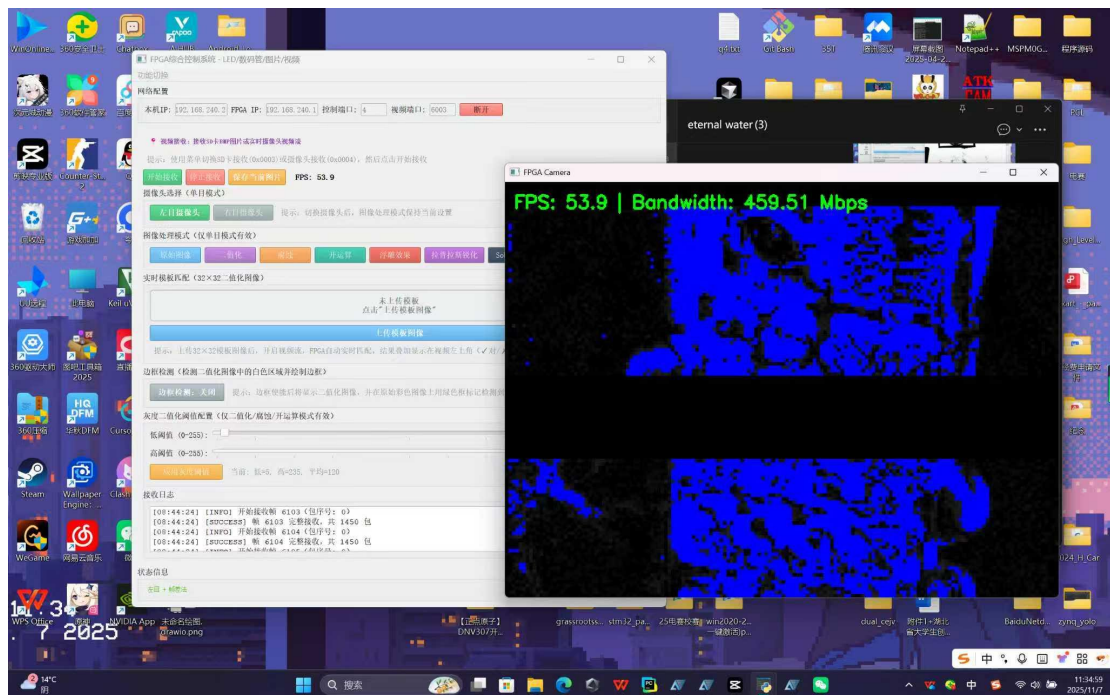
3.2.3 二值化



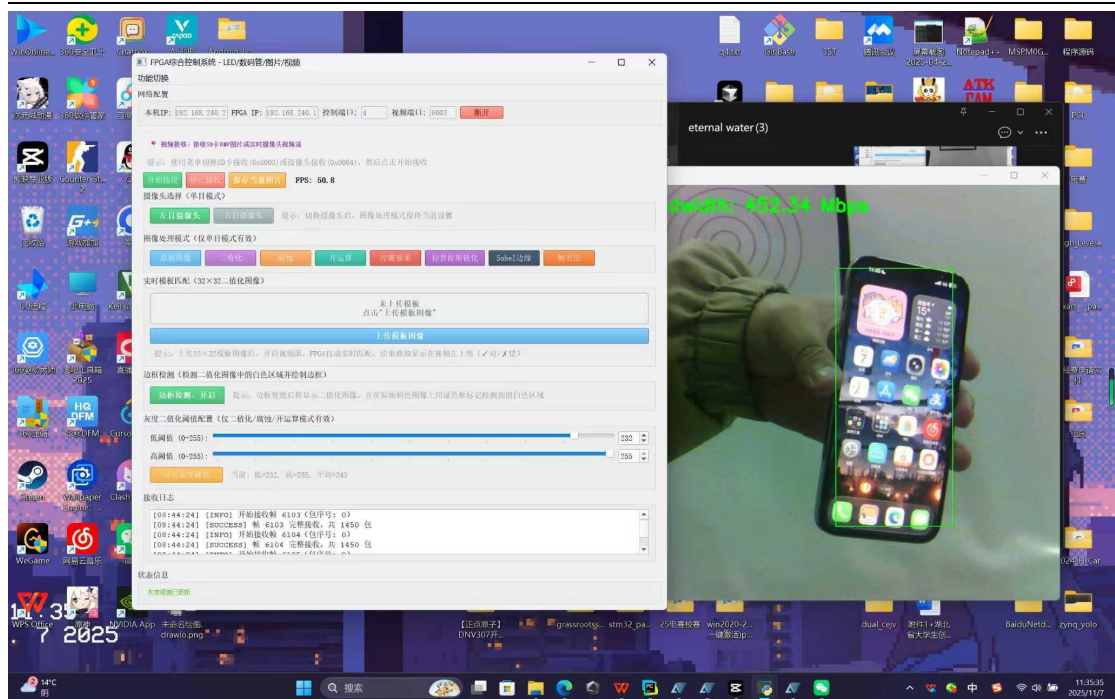
3.2.3 拉普拉斯锐化



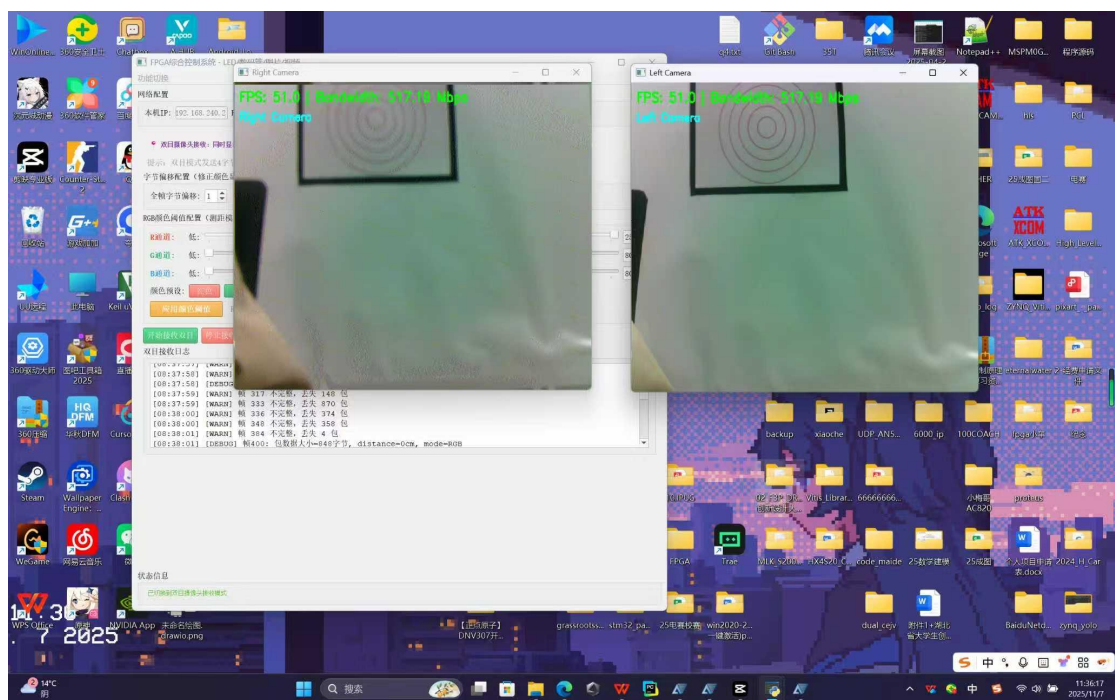
3.2.4 Sobel 边缘检测



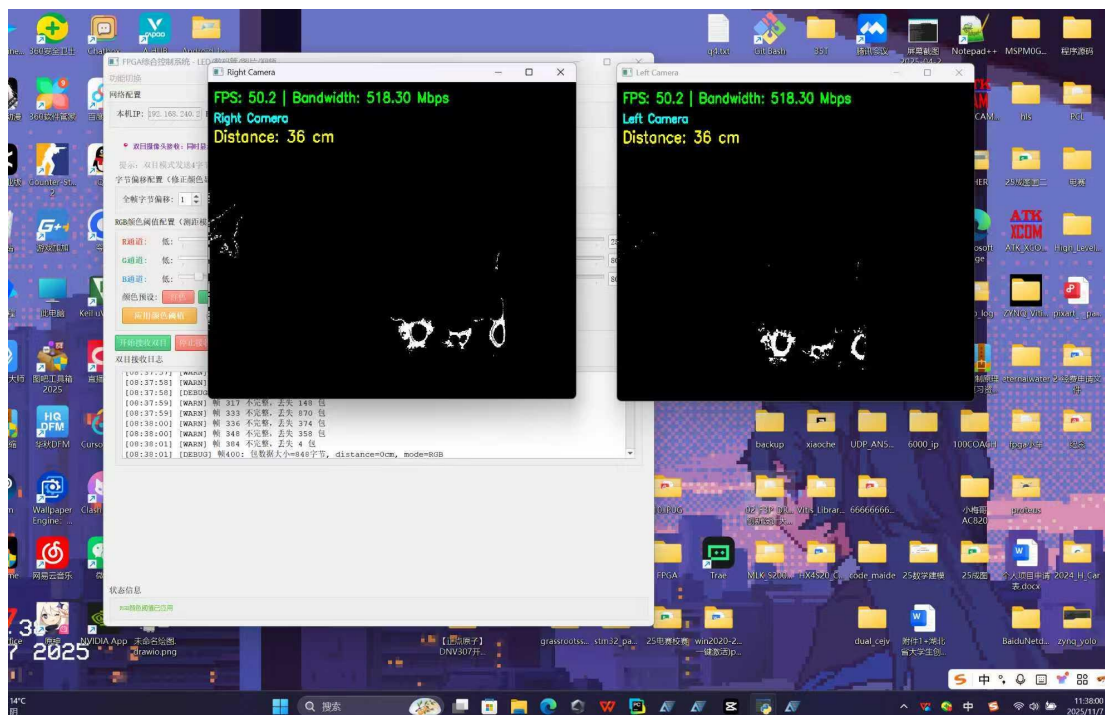
3.2.5 帧插法



3.2.6 二值化边缘检测



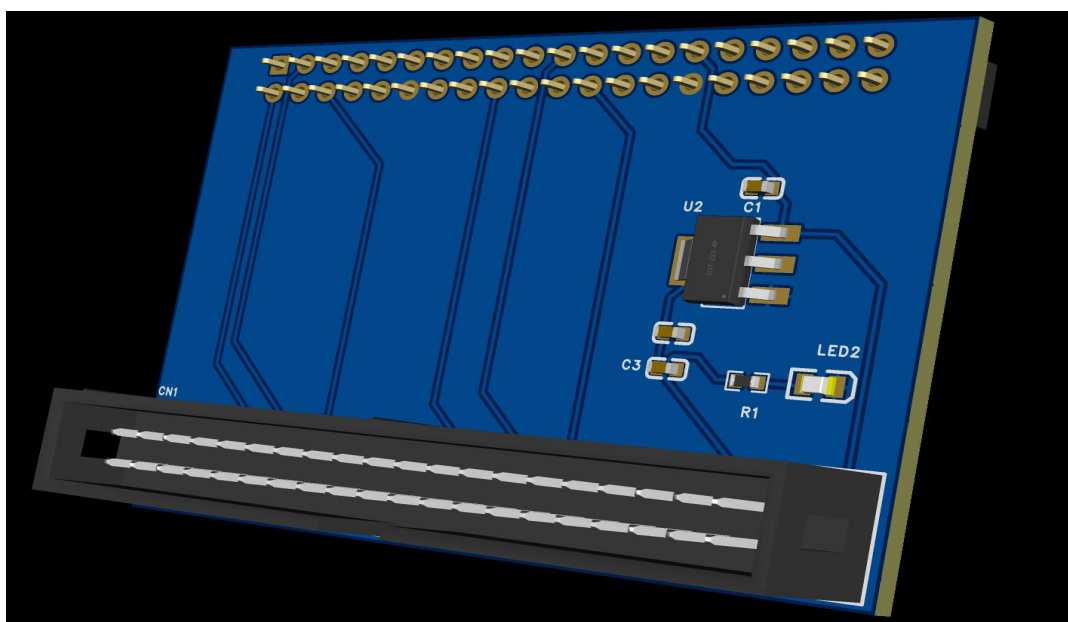
3.2.6 上位机接收双目图像



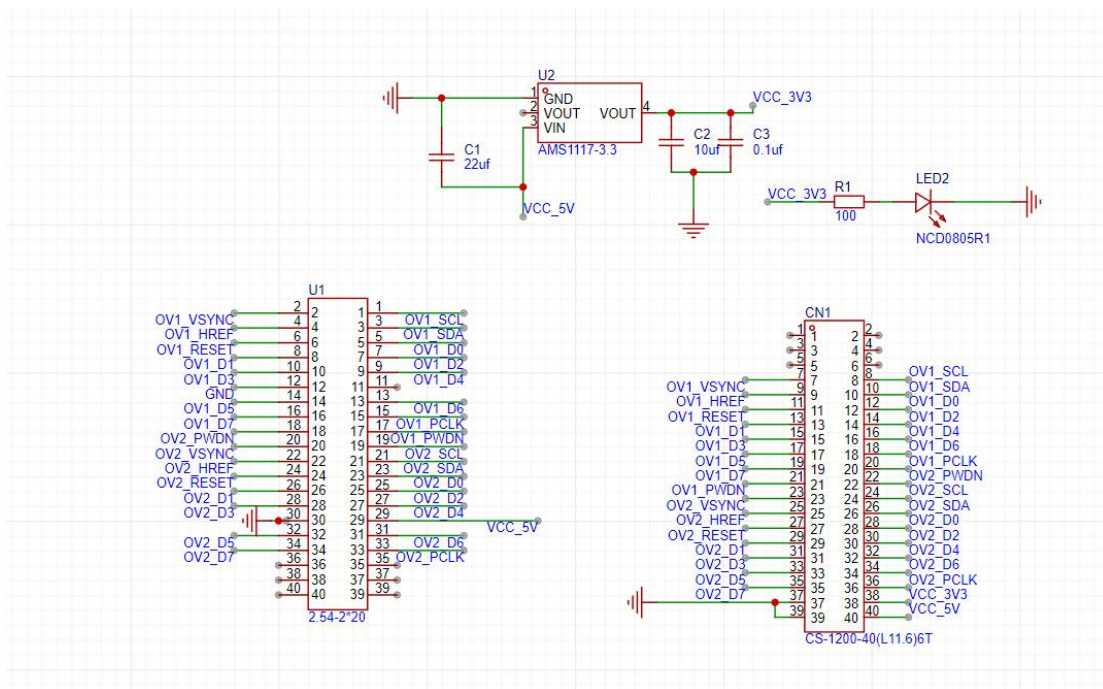
3.2.7 双目测距

3.2.1 电路成果

由于开发板上没有输出 3.3 伏的供电所以我们制作设计了一款摄像头扩展模块。

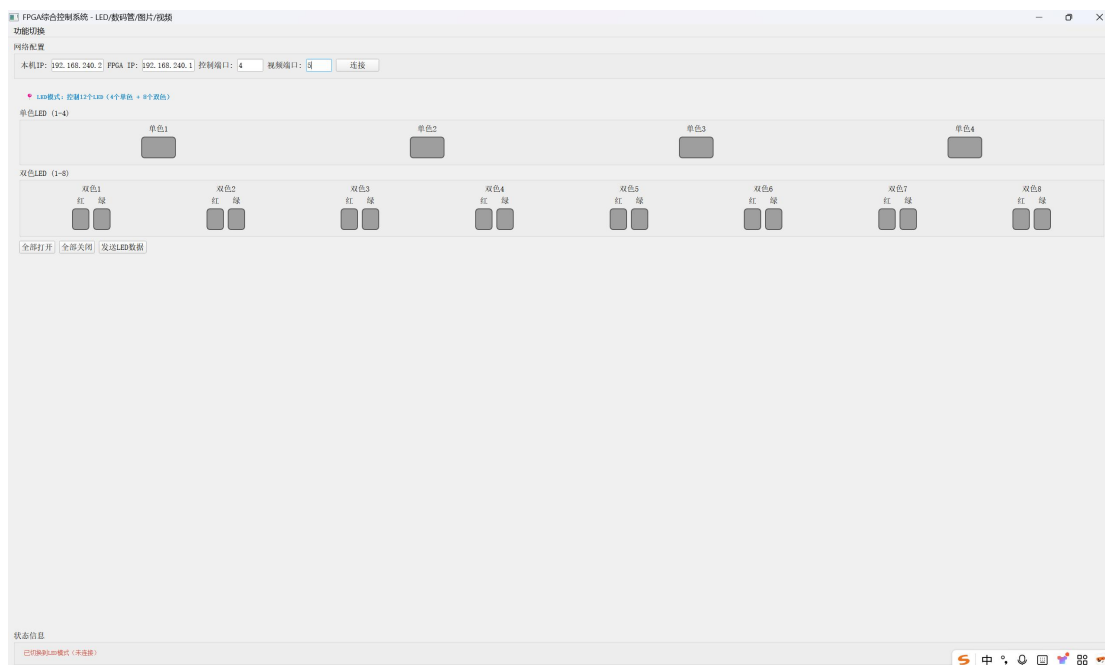


3.2.1.1 ov5640摄像头扩展板

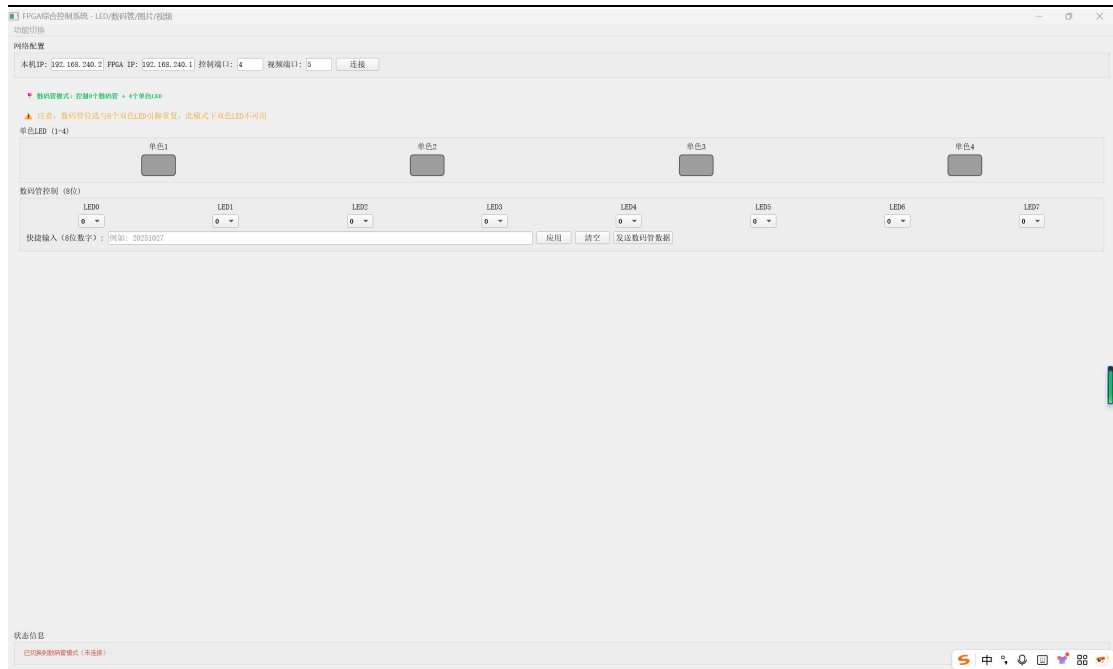


3. 2. 1. 2摄像头扩展板引脚图

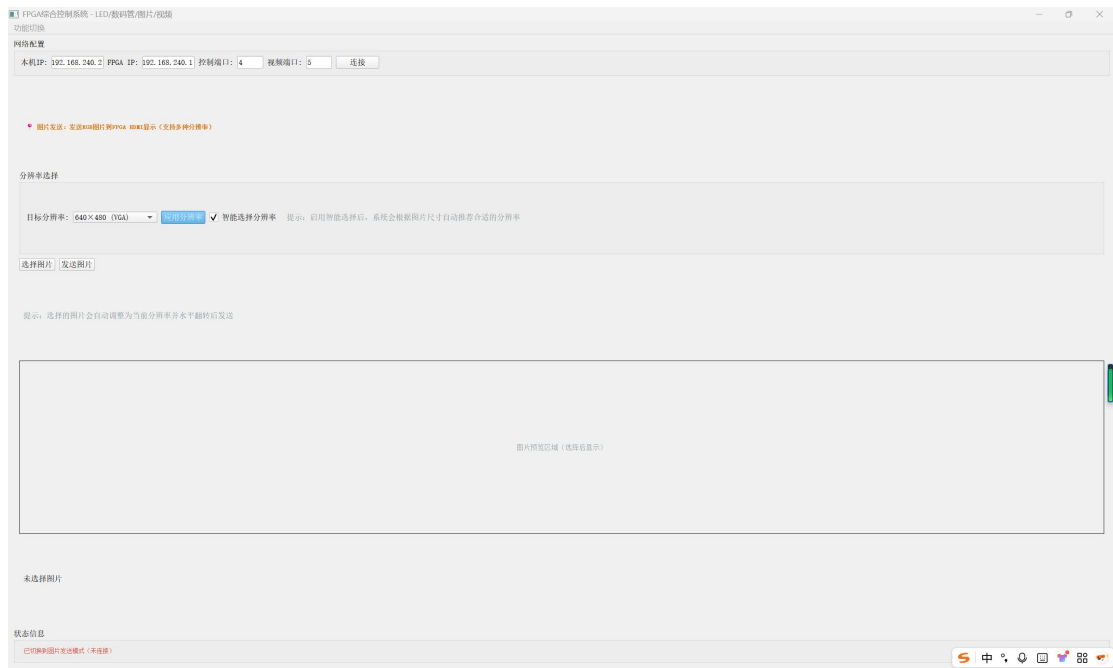
3.2.2 软件成果



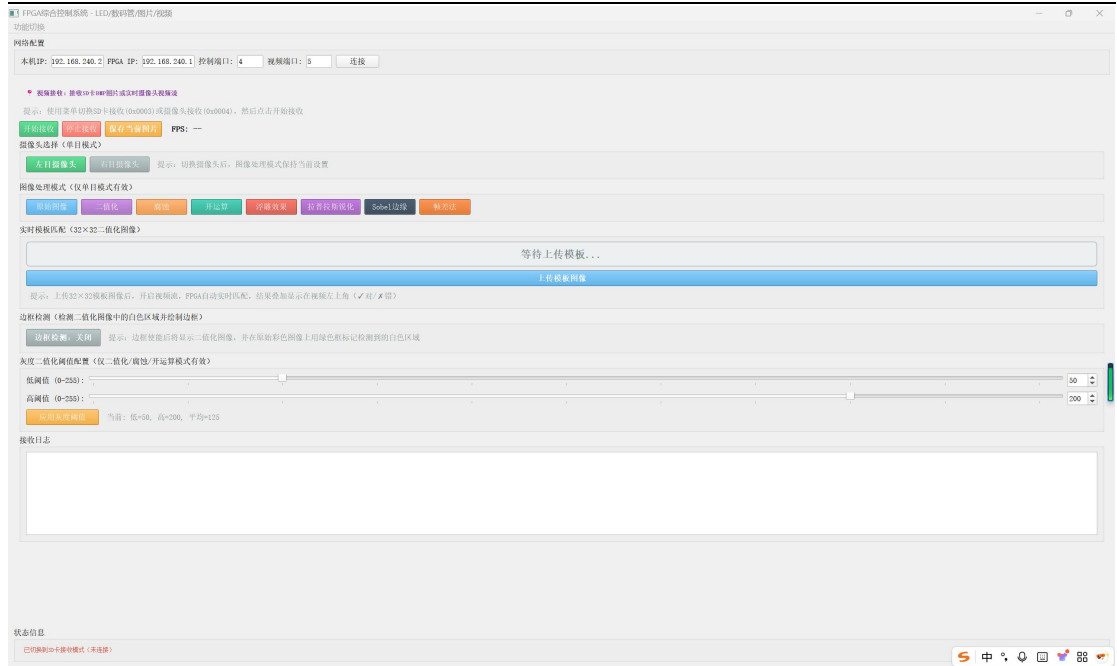
3. 2. 2. 1上位机的展示(步骤一)



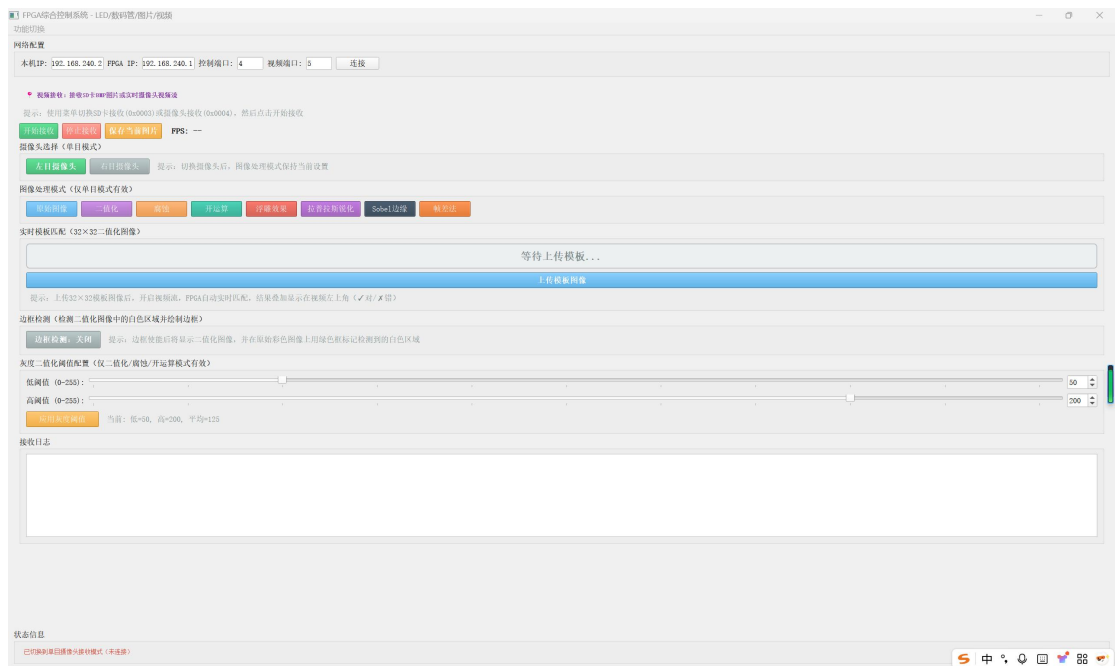
3. 2. 2. 2上位机的展示(步骤二)



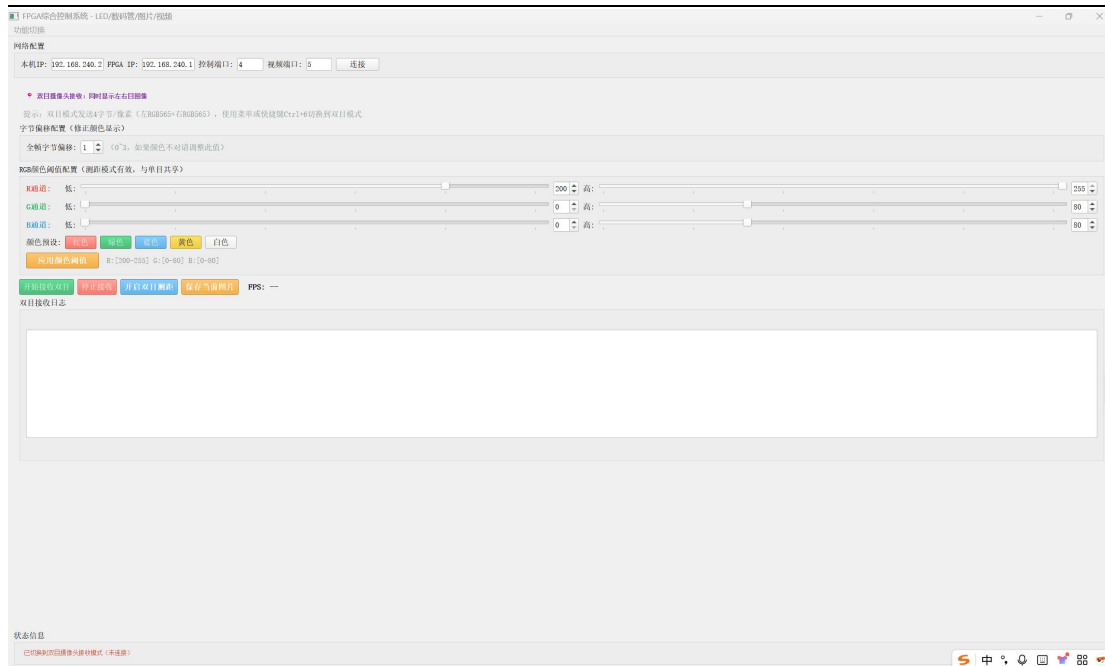
3. 2. 2. 3 上位机的展示(步骤三)



3.2.2.4 上位机的展示(步骤四)

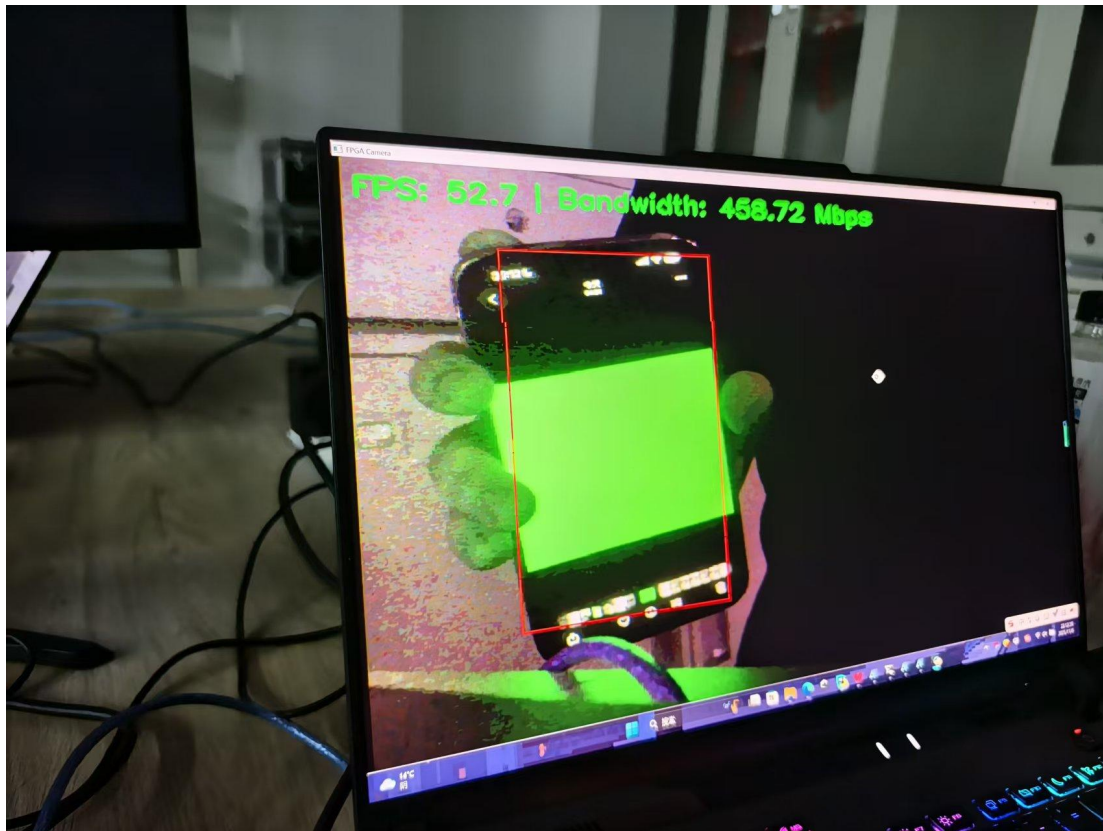


3.2.2.5 上位机的展示(步骤五)

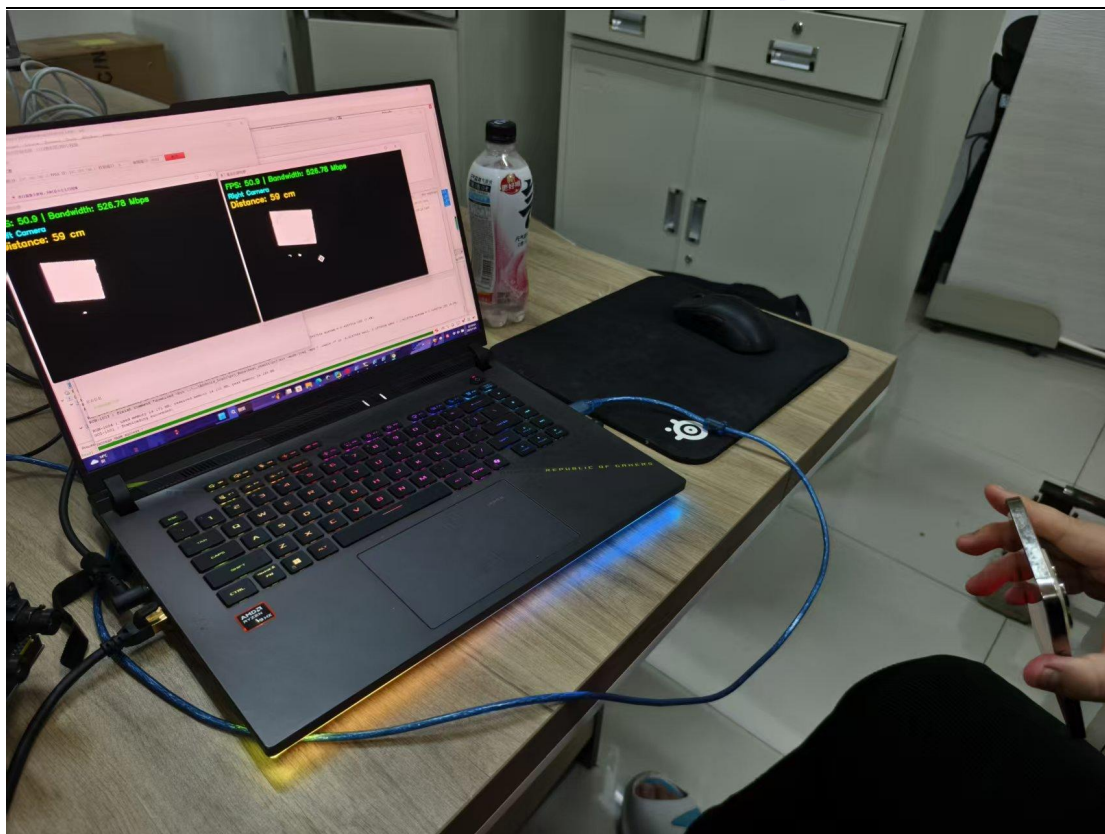


3.2.2.6 上位机的展示(步骤六)

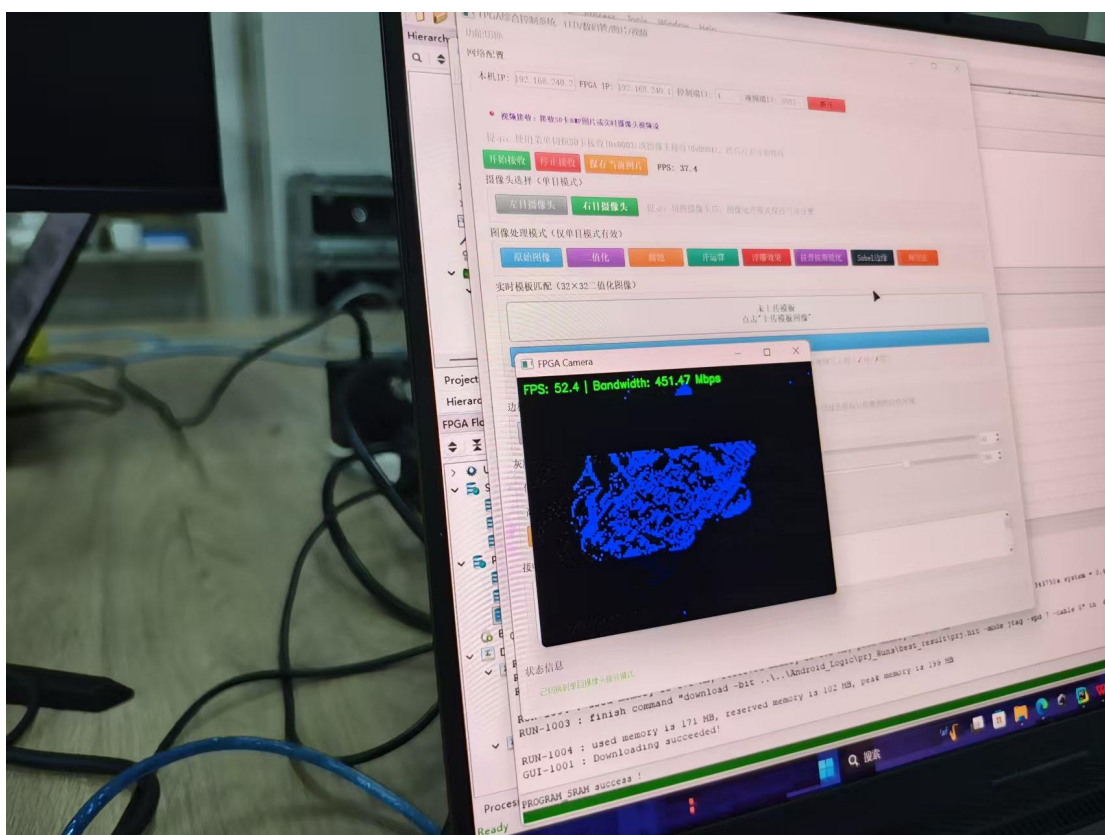
3.3 特征成果



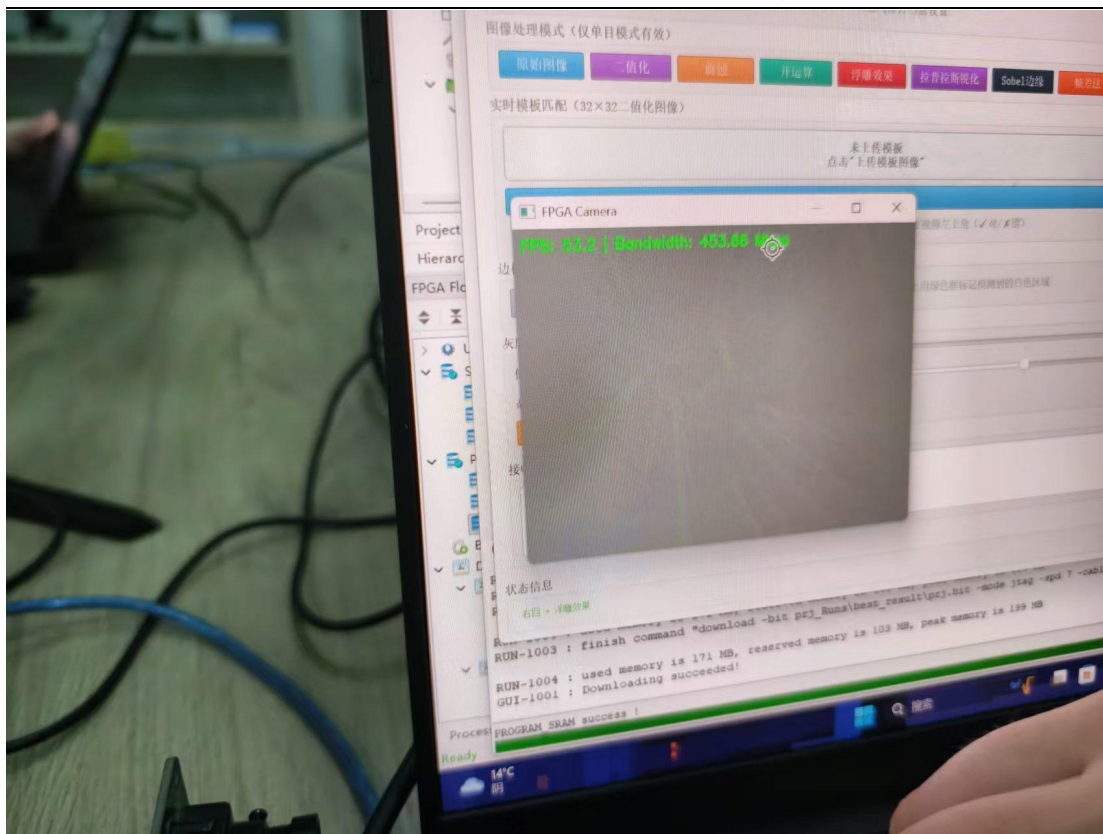
3.3.1 边缘检测



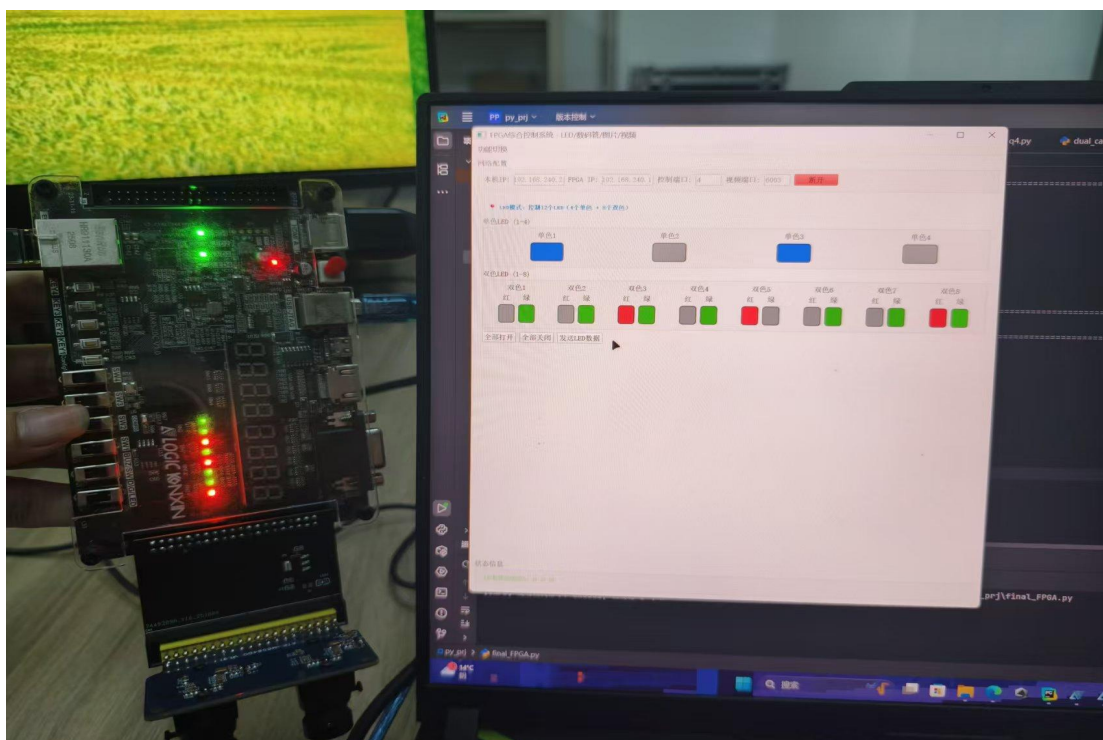
3.3.2. 双目测距



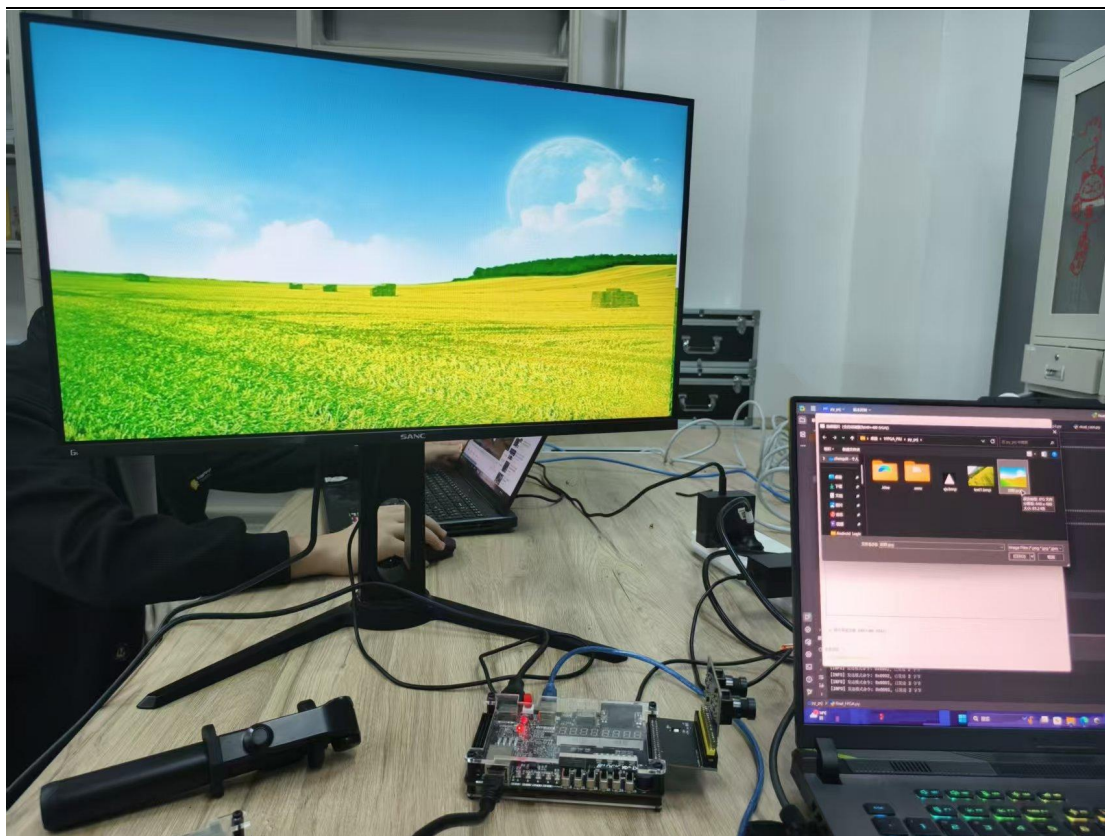
3.3.3 帧查法



3.3.4 浮雕效果

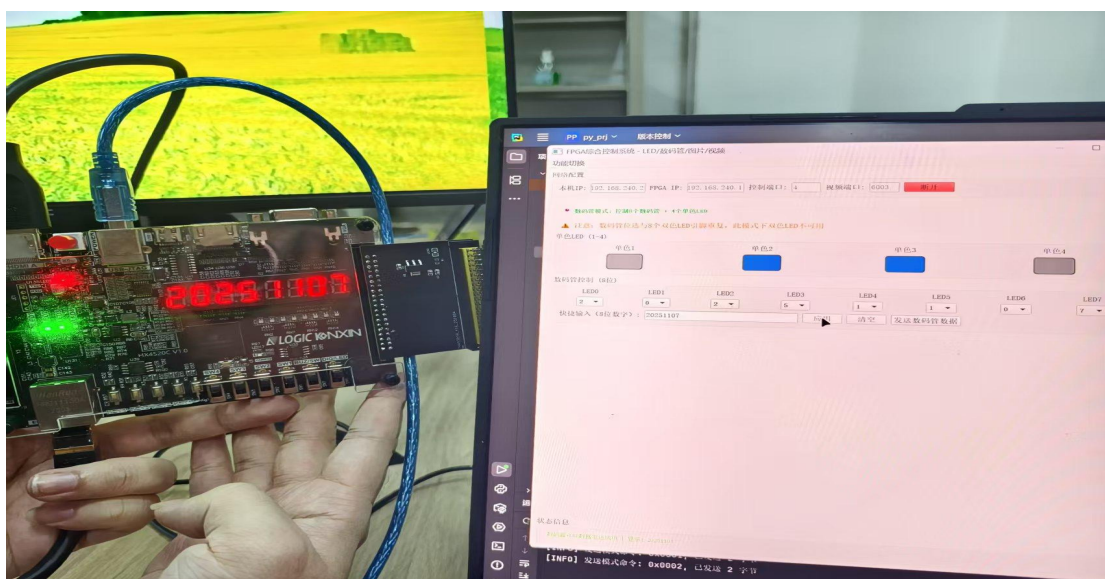


3.3.5 上位机控制 LED 灯（步骤一）



3.3.6 上位机控制 LED 灯（步骤二）

3.3.6 上位机控制 LED 灯（步骤二）



3.3.7 上位机控制 LED 灯（步骤三）

第四部分 总结

4.1 可扩展之处

4.1.1 图像处理能力扩展

当前支持二值化、腐蚀、膨胀等基础算法，可扩展边缘检测、特征提取、目标识别等高级算法，以及 3D 图像处理和实时视频分析功能。

4.1.2 网络通信协议扩展

现有 UDP 协议栈可扩展为支持 TCP/IP、HTTP、WebSocket 等多协议，并可集成 WiFi、蓝牙等无线通信模块。

4.1.3 存储系统扩展

当前 SDRAM 可扩展为 eMMC、SSD 等大容量存储，并实现文件系统、缓存策略和数据安全机制。

4.1.4 显示输出扩展

单一 HDMI 输出可扩展为多屏显示系统，支持 3D 显示、AR/VR 及交互界面。

4.1.5 智能控制扩展

可集成 AI 决策模块，实现机器学习、深度学习算法，提供智能分析和自适应

应控制能力。

4.2 心得体会

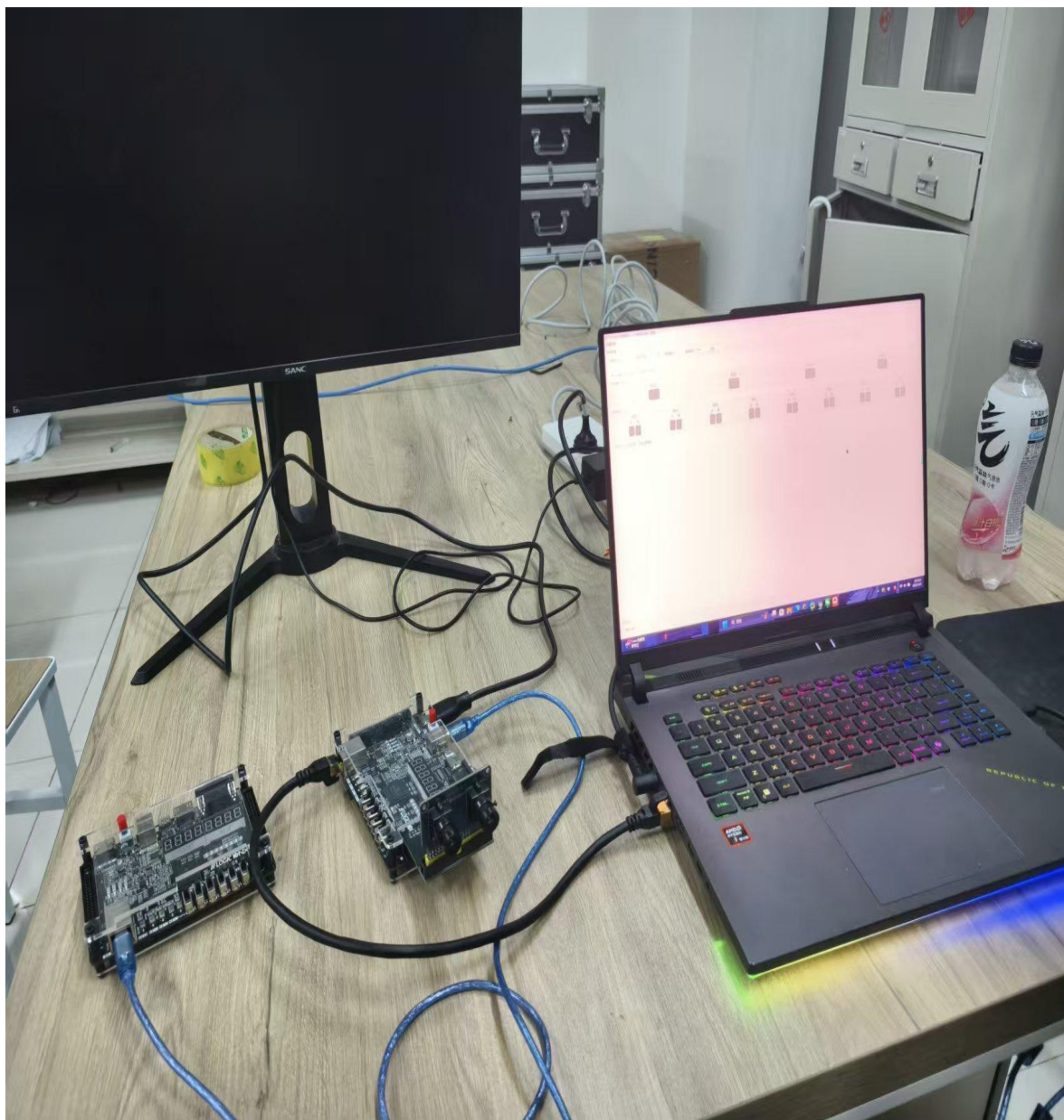
参与项目的过程，是我们技术成长和思维转变的重要阶段。我们不仅掌握了 FPGA 设计的核心技术，更重要的是学会了如何从系统角度思考问题，如何在资源约束下寻找最优解决方案。模块化设计让团队理解了复杂系统管理的精髓，调试与优化过程培养了细致入微的观察力，团队协作则让我们认识到沟通和协调的重要性。这些经验不仅适用于 FPGA 开发，也对大家的未来的技术生涯产生了深远影响。回顾整个项目，最宝贵的收获不是技术知识的积累，而是解决问题能力的提升和系统思维的建立。在未来的工作中，我们将继续保持这种学习和探索的态度，不断挑战自我，追求技术的卓越与创新。

第五部分 参考文献

[1]张宏宇.基于 ZYNQ 的双目立体视觉系统设计 [D].中北大学,2023.DOI:10.27470/d.cnki.ghbgc.2023.000459.

第六部分 附录

6.1 系统器件图



6.1.1 系统全貌图



6.1.HX4S20C 开发板（正面）



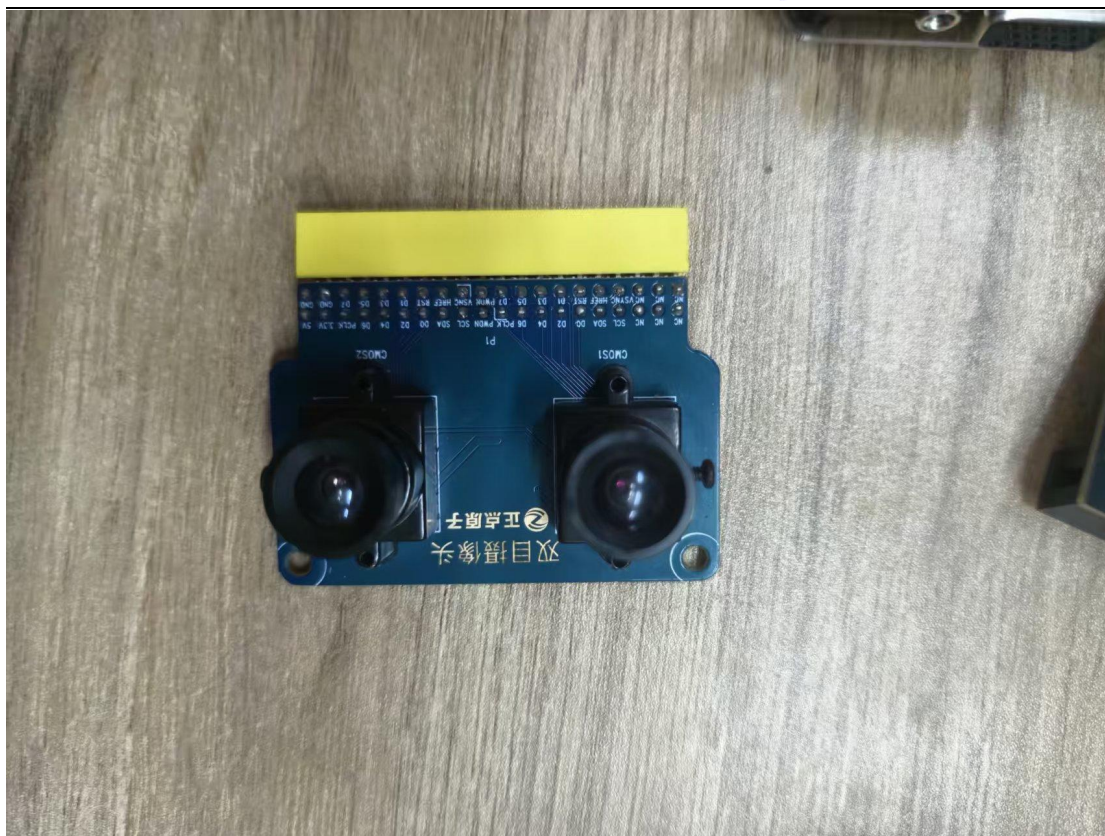
6.1.3 HX4S20C 开发板板（反面）



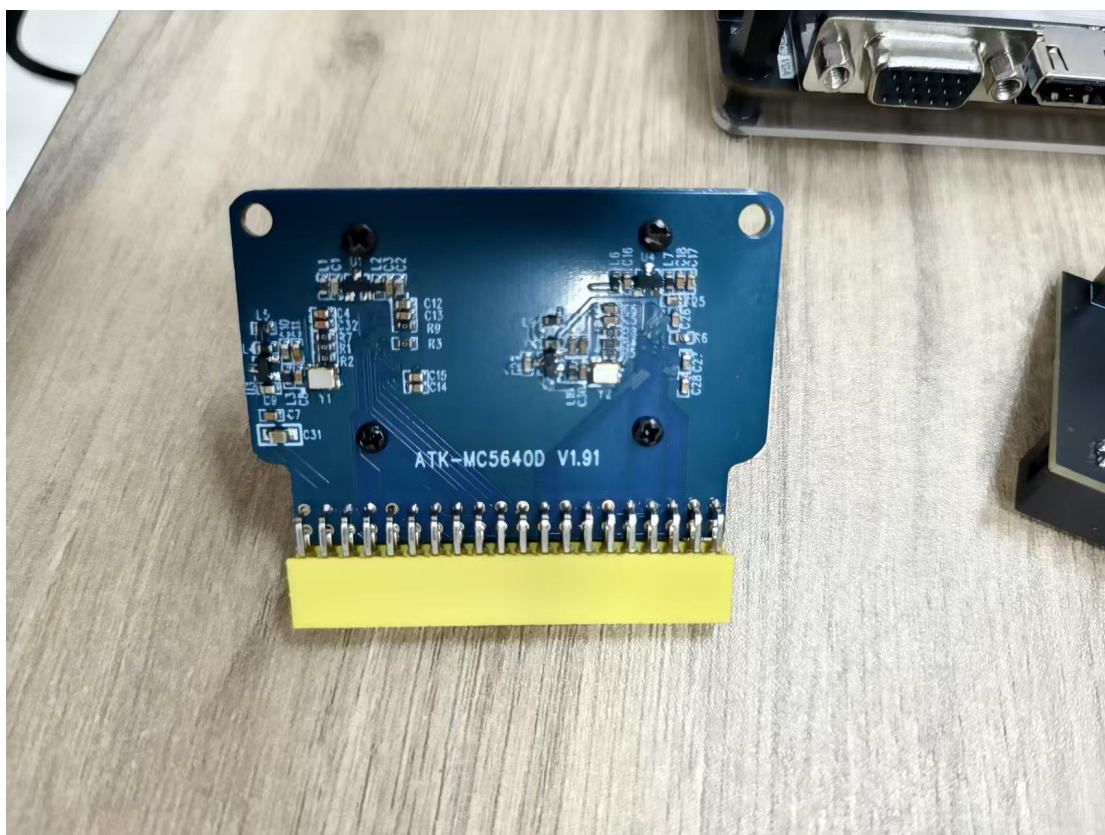
6.1.4 摄像头扩展板（正面）



6.1.5 摄像头扩展板（反面）



6.1.5 ov5640 双目摄像头（正面）



6.1.6 ov5640 双目摄像头（反面）